CROSS-LANGUAGE PLAGIARISM DETECTION IN ACADEMIC PROGRAMMING CODES

Report By

SHAKEEL AHAMED SHAJAHAN | CB009882

BSc (Hons) Computer Science

Asia Pacific Institute of Information Technology (APIIT) / University of Staffordshire

Supervisor

Ms. Adshayani Pirapaharan

June 2026

DECLARATION

1. I know that plagiarism means taking and using the ideas, writings, works or inventions of another as if they were one's own. I know that plagiarism not only includes verbatim copying, but also the extensive use of another person's ideas without proper acknowledgement (which includes the proper use of quotation marks). I know that plagiarism covers this sort of use of material found in textual sources and from the Internet.

2. I acknowledge and understand that plagiarism is wrong.

3. I understand that my research must be accurately referenced. I have followed the rules and conventions concerning referencing, citation and the use of quotations as set out in the Departmental Guide.

4. This assignment is my own work. I acknowledge that copying someone else's assignment, or part of it, is wrong, and that submitting identical work to others constitutes a form of plagiarism.

5. I have not allowed, nor will I in the future allow, anyone to copy my work with the intention of passing it off as their own work.

Name – Shakeel Ahamed Shajahan

Student – CB009882

Signed - 

Date – 16th June 2026

ABSTRACT

Programming assignment plagiarism is a growing problem for educational institutions, particularly because students increasingly are using cross-language code reuse to evade traditional plagiarism detection methods. Current methods like MOSS and JPlag are limited to a single programming language paradigm and so are unable to detect semantically equivalent submissions in different languages. This restriction results in high false negative rates and many plagiarized materials that are not detected. With the expansion of multilingual programming education, courses are now being implemented that teach Python, Java, and C# at the same time, and a language-agnostic approach to detecting programming language is essential. This is the key to maintaining academic integrity and equitable assessment in a wide range of programming contexts.

In this paper, we present a cross-language plagiarism detection system that is developed for Python, Java, and C# by using multi-stage pipeline to detect deep semantic similarity beyond lexical or syntax similarity. The source code is first processed by parsing and normalizing it with the Tree-sitter-based Abstract Syntax Tree (AST) extraction, which removes language-specific syntactic noise. The normalized representations are then converted to language-agnostic semantic embeddings using a pretrained cross-modal code representation model, UniXcoder. This is then followed by training a Siamese neural network for pairwise similarity classification on these embeddings. An Information Retrieval (IR) similarity mechanism is used as a complementary scoring layer that further fleshes out confidence in detections. The system is tested on a cross-language dataset based on a published Java programming assignment dataset, complemented with semantically similar implementations in C# and Python, and rolled out as a complete web application that can be used by educators.

Results from the experimental evaluation show that the proposed system shows good and steady detection performance in all three language pairs. The system achieves an accuracy of 91.71%, precision of 93.42%, recall of 88.83%, F1 score of 91.06% and an AUC-ROC of 0.9683, which confirms its excellent discriminative power between plagiarized and original documents.

ACKNOWLEDGEMENTS

I would like to express my gratitude to my project supervisor Ms. Adshayani Pirapaharan for her guidance, constant encouragement and constructive feedback. They shared their expertise and knowledge of the local area and were key to the success of this project.

I would also like to express my gratitude to the Module Leader Mr. Guhanathan Poravi for laying the foundation as well as providing me with the tools to complete this research.

There is special thanks to my peers and colleagues for their exciting and stimulating discussions, technical assistance and spirit of cooperation during the development and research challenges. Finally, I would like to express my sincere gratitude to my family, friends and all people who have been patient, understanding and supportive of me during my educational experience. Their consistent encouragement has been a great motivator of mine.

With regards,

Shakeel Ahamed Shajahan

CB009882

TABLE OF CONTENTS

Declaration ii

Abstract iii

Acknowledgements iv

Table of Contents v

List of Figures x

List of Tables x

List of Abbreviations xi

Chapter 1 – introduction 1

1.1. Chapter Overview 1

1.2. Problem Background 1

1.2.1. Context of the Research Domain 1

1.2.2. The Shift to Multi-Language Curricula 2

1.2.3. AI-Assisted Code Generation and Obfuscation 2

1.3. Problem Definition 3

1.3.1. Problem Statement 4

1.4. Research Motivation 4

1.5. Research Gap 4

1.6. Contribution to the Body of Knowledge 6

1.6.1. Contribution to the Research Domain 6

1.6.2. Contribution to the Problem Domain 6

1.7. Research Challenges 7

1.8. Research Questions 8

1.9. Research Aim 8

1.10. Research Objectives 8

1.11. Chapter Summary 9

CHAPTER 2 – LITERATURE REVIEW 11

2.1. Chapter Overview 11

2.2. Concept Map 11

2.3. Problem Domain Review 12

2.3.1. Introduction to the Domain. 12

2.3.2. Key Challenges 13

2.3.3. Existing Frameworks, Standards, and Domain Approaches 15

2.3.4. Proposed Architecture and Positioning 16

2.4 Technological Review 17

2.4.1. Information Retrieval Similarity Pre Filtering 17

2.4.2. Tree-sitter for Multi-Language AST Normalization 18

2.4.3. UniXcoder for Cross-Language Code Embeddings 18

2.4.4. Siamese Neural Networks for Similarity Learning 19

2.4.5. Contrastive Learning for Similarity Optimization 20

2.5. Existing Work 20

2.5.1. Single Language Detection Tool 20

2.5.2. Cross-Language Detection Approaches 21

2.5.3. Deep Learning Approaches 22

2.6. Evaluation and Benchmarking 23

2.6.1. Datasets 23

2.6.2. Evaluation Metrics 24

2.6.3. Benching Limitations 25

2.7. Chapter Summary 25

CHAPTER 3 – METHODOLOGY 27

3.1. Chapter Overview 27

3.2. Research Methodology — Saunders' Research Onion 27

3.2.1. Philosophy - Positivism 27

3.2.2. Approach - Deductive 28

3.2.3. Strategy - Experimental 28

3.2.4. Methodological Choice - Mono-Method Quantitative 28

3.2.5. Time Horizon — Cross-Sectional 29

3.2.6. Data Collection and Analysis — Controlled Experimentation with Quantitative Analysis 29

3.3. Development Methodology 30

3.3.1. Methodology Selection - Agile Iterative Development 30

3.3.2. Comparison with Alternatives 30

3.3.3. Sprint Structure 31

3.4. Project Management Methodology 31

3.4.1. Schedule 32

3.4.2. Deliverables and Dates 33

3.4.3. Resources 34

3.4.4. Data Requirements 36

3.4.5. Risk and Mitigation 37

3.5. Chapter Summary 39

CHAPTER 4 – LEGAL, ETHICAL, SOCIAL AND PROFESSIONAL ISSUES (LESP) 40

4.1 Chapter Overview 40

4.2 BCS Code of Conduct 40
4.3 Social Issues 41
4.4 Legal Issues 43
4.5 Ethical Issues 44
4.6 Professional Issues 45
4.7 Chapter Summary 45
CHAPTER 5 – IMPLEMENTATION / EXPERIMENTAL SETUP 47
5.1. Chapter Overview 47
5.2. Technology Selection 47
5.2.1. Programming Language 47
5.2.2. Libraries 47
5.2.3. Frameworks 48
5.3. Data Selection 50
5.4. Implementation of Core Features 51
5.4.1. IR Similarity Prefilter 51
5.4.2. Tree-sitter AST Normalization 51
5.4.3. UniXCoder Semantic Embedding 52
5.4.4. Siamese Neural Network Training 53
5.5. User Interface. 54
5.6. Chapter Summary 55
CHAPTER 6 – RESULTS AND EVALUATION 56
6.1. Chapter Overview 56
6.2. Quantitative Results 56
6.2.1. Core Performance Metrics 56

6.2.2 Confusion Matrix Analysis 57
6.3. Qualitative Analysis 57
6.3.1. System Output and Web Application Interface 57
6.4. Ablation Study 58
6.5. Baseline and Comparison 59
6.6. Discussion 60
6.6.1. Research Question Responses 60
6.6.2. Strengths 61
6.6.3. Limitation and Unexpected Outcome 62
6.6.4. Practical Implementation 62
6.7. Chapter Summary 63
CHAPTER 7 – CONCLUSION AND FUTURE WORK 64
7.1. Chapter Overview 64
7.2. Similar Findings 64
7.2.1. Research Questions 64
7.3 Contributions 65
7.3.1 Contributions to the Research Domain 65
7.3.2 Contributions to the Problem Domain 66
7.4 Limitations 67
7.5 Future Work 67
7.6 Contribution to the Body of Knowledge 69
7.6.1. Research Domain 69
7.6.2. Problem Domain 69
7.7 Concluding Remarks 70

References I

Appendices III

Gantt Chart III

LIST OF FIGURES

Figure 2.0.1: Concept Map of Cross-Language Plagiarism Detection 12

Figure 2.0.2: Proposed Conceptual Architecture 16

Figure 5.0.1: IR Similarity Implementation 51

Figure 5.0.2: Tress-sitter AST Normalization Implementation 52

Figure 5.0.3: UniXcoder Implementation 53

Figure 5.0.4: Siamese Neural Network 53

Figure 5.0.5: File Upload Page 54

Figure 5.0.6: Result Output 55

Figure 6.0.1: Quantitative Data Results 56

Figure 6.0.2: Confustion Matixs 57

Figure 6.3: System Output and Interface 58

LIST OF TABLES

Table 3.0.1: Saunders Research Onion Summary 30

Table 3.0.2: Project Schedule Summary (see Appendix II for full Gantt Chart) 33

Table 3.0.3: Project Deliverables and Due Dates 33

Table 3.0.4: Hardware Resources 34

Table 3.0.5: Software Resources..... 35

Table 3.0.6: New Technical Skills Table..... 36

Table 3.0.7: Risks & Mitigations 39

Table 5.0.1: Technology Report Table 50

Table 6.0.1: Ablation Study Summary..... 59

Table 6.2: Baseline and Comparison summary..... 59

LIST OF ABBREVIATIONS

API - Application Programming Interface

APIIT - Asia Pacific Institute of Information Technology

AST - Abstract Syntax Tree

AUC-ROC - Area Under the Receiver Operating Characteristic Curve

BCS - British Computer SocietyBM25

CFG - Control Flow Graph

CodeBERT - Code Bidirectional Encoder Representations from Transformers

CodePTM - Code Pre-Trained Model

CPU - Central Processing Unit

DPIA - Data Protection Impact Assessment

ESA - Explicit Semantic Analysis

ESA-GST - Explicit Semantic Analysis and Scored Greedy String Tiling

F1 Score (Harmonic Mean of Precision and Recall)

FN - False Negative

FP - False Positive

GDPR - General Data Protection Regulation

GLUE - General Language Understanding Evaluation

GPU - Graphics Processing Unit

GST - Greedy String Tiling

ICO - Information Commissioner's Office

IDE - Integrated Development Environment

IP - Intellectual Property

IR- Information Retrieval

LESP - Legal, Ethical, Social and Professional

LINQ- Language Integrated Query

LMS - Learning Management System

ML - Machine Learning

MOSS - Measure of Software Similarity

MVP - Minimum Viable Product

NLP 0 Natural Language Processing

PDG - Program Dependency Graph

PRINCE2 - Projects in Controlled Environments 2

REST - Representational State Transfer

RO - Research Objective

RQ - Research Question

SHA-256 - Secure Hash Algorithm 256-bit

SMART - Specific, Measurable, Achievable, Relevant, Time-bound

SOCO - Source Code Re-use (Dataset)

SR - S - Software Requirements Specification

TF-IDF - Term Frequency – Inverse Document Frequency

TN - True Negative

TP - True Positive

UI - User Interface

UK - United Kingdom

UK GDPR - United Kingdom General Data Protection Regulation

UniXcoder - Unified Cross-Modal Pre-training for Code Representation

URL - Uniform Resource Locator

YAP - Yet Another Plague

CHAPTER 1 – INTRODUCTION

1.1. Chapter Overview

This chapter provides a basic context for the research of cross-language plagiarism detection in academic programming codes. It starts by placing the work in the context of computer science education and academic honesty and then focuses on the problem of finding semantically similar code across programming languages. The chapter then sets up and formally defines the problem, documents the need for research, and states the contributions the research will make to the research problem. The research challenges, questions, aim and SMART objectives are then presented and followed by a summary that leads into an introduction for the Literature Review in Chapter 2. This chapter introduces the reader to the study, its significance, and its purpose in keeping with the TTT principle.

1.2. Problem Background

1.2.1. Context of the Research Domain

Programming exercises are fundamental to computer science education and are widely used to evaluate students' problem-solving abilities, algorithmic thinking, and software development skills. Academic honesty is, in this context, not just a disciplinary issue, it is an integral part of the system of ensuring the assessment results are a true reflection of student competence and that the qualifications are of credible value. In programming classes, however, the problem of plagiarism has become increasingly complex and pervasive. While textual plagiarism is easier to detect, source code plagiarism may appear as structural, semantic or logic-preserving copying – where very little, if any, surface-level similarity exists to the original submission – can occur (Maertens et al., 2022).

This is compounded by the size of today's programming modules. Manual assessment of submissions is not feasible or manageable in large groups, imposing an unrealistic burden on the academic faculty and risk of over-detection and under-detection (Aniceto, Holanda and Da Silva, 2023). To address this, institutions have turned to implementing automated detection methods like MOSS and JPlag, which are typically used to compare text or monolingual code, but are

unable to detect plagiarism that spans across programming language boundaries (Maertens et al., 2022). With the increasing sophistication of plagiarism techniques, these old systems are questioned increasingly.

1.2.2. The Shift to Multi-Language Curricula

Doing so, contemporary computer science curricula often introduce students to more than one programming language as a part of different modules and academic years in the curriculum. For example, as part of the introductory problem-solving module, students might be taught the programming language Python, and as part of the object-oriented programming module, students might be taught the programming language Java, and as part of enterprise software development, students might be taught the language C#. Although this multi-language approach fits better with the standard software development practice, it also creates a great vulnerability in plagiarism detection – a student can easily come up with a solution in another language, while it is already known in the first language, consequently all the monolingual plagiarism detection tools can be circumvented (Foltýnek et al., 2020).

This form of evasion is hard to counter because the same algorithmic structure can be implemented using syntactically unrelated structures in different languages. Conventional token-matching and AST-comparison methods fail to capture structural differences between different Abstract Syntax Trees (ASTs) that can represent the same computation, for example in Python using a list comprehension, in Java using a call to the Stream API, and in C# using a LINQ expression (Maertens et al., 2022). The basic detection problem thus becomes one of language-independent equivalence of the underlying computational problem described by two submissions, not syntactic overlap. Doing this demands that representations be sensitive to the meaning of programs, not just their surface syntax, which existing tooling cannot do at scale.

1.2.3. AI-Assisted Code Generation and Obfuscation

In addition to cross-language translation, the real-world scenario of plagiarism has changed drastically with the advancement of AI-driven coding tools and open-source repositories. The students have, now, the ability to produce functional solutions in seconds and then obfuscate the code with renaming of identifiers, reordering of independent statements, changing line spacing, and changing the control flow, all to the end of concealing the origin of the code (Sağlam and

Schmid, 2025). These are low effort strategies that generate submissions that maintain the same logical behavior as the original but also provide enough surface variation to avoid detectors based on lexical or token similarity (Santos and Ferreira, 2020).

At the same time, academic workflows require that plagiarism detection be accurate, as well as be fast. At the same time, academic workflows are demanding that plagiarism detection be accurate, as well as be fast. The MOSS has quadratic time complexity $O(n^2)$, which is impractical at scale, because of the batch processing methods (Wan, Liu and Gao, 2018). Combined, these forces pervasive sophistication of obfuscation, the ability to evade across languages, and the computational limitations of existing tools set the boundaries of the practical context in which this research fits. The detection system to identify plagiarism at the semantic level in multiple languages, accurate and feasible in academic settings is needed.

1.3. Problem Definition

While there have been significant developments in source code similarity analysis, there is not yet a system that can reliably identify plagiarism that spans programming languages that is also able to resist logic-preserving obfuscation. Existing systems like MOSS, JPlag and Dolos focus on the single language paradigm and depend on lexical fingerprinting and syntactic token matching, which are both structurally deficient when the surface syntax changes completely, as is the case in cross-language reuse cases (Maertens et al., 2022; Foltýnek et al., 2020). Theoretically, semantic embedding approaches with pre-trained code models have the potential to deliver a wide range of benefits, however, existing implementations are lacking in the following areas: a standardized cross-language evaluation set, real-time inference capability, and deployable system implementations that are suitable for academic use (Ebrahim and Joy, 2023).

This study is restricted to three languages: Java, Python and C, because these were chosen due to their widespread use in undergraduate CS curricula. Examples of what is not included are natural language plagiarism, binary or compiled code comparison, and integration with live learning management systems.

1.3.1. Problem Statement

There is no existing reliable real-time system that detects cross-language source-code plagiarism among Java, Python and C with the robustness of being able to stay immune to logic-preserving structural and textual obfuscation in academic programming submissions.

1.4. Research Motivation

There is a clear failure mode with all existing plagiarism detectors that they are not architecturally capable of bridging the language barrier. Tools based on tokens have language-specific syntax in their foundation logic of comparison and thus a functionally equivalent submission in a different language is considered completely different (Maertens et al., 2022). While some studies attempt to translate code into a common vector space irrespective of the languages such as UniXcoder (Guo et al., 2022) and Fokam and Ajoodha (2021) no previous work has integrated both language embeddings with a Siamese network in a single model and tested it on a structured cross-language academic dataset that spans three languages, namely, Java, Python, and C.

Plagiarism in assessed work in programming affects the integrity of the computer science qualification and gives an unfair advantage to those who work hard on their own solutions. There is an increasing disparity between the ease of submitting plagiarized content and its difficulty to detect as AI-generated code becomes more prevalent (Sağlam and Schmid, 2025). Academic institutions need tools to be technically capable and practically deployable, and this research directly addresses this need by providing a complete end-to-end, web-based application along with the detection model.

As a computer science student, I have seen the shortcomings in current systems in an educational environment, and I also have a personal interest in building a system that is both fair and accurate and can be used by teachers without any particular expertise in machine learning.

1.5. Research Gap

GAP 01 – Absence of effective cross-language detection system

The current plagiarism detection system programs are basically monolingual. MOSS, JPlag, Dolos are tools that extract token sequences or AST fingerprints that are tightly coupled to the grammar of a specific programming language, and which cannot compare submissions across language boundaries (Maertens et al., 2022). Cross-language detection was introduced by Foltýnek et al. (2020) with an Explicit Semantic Analysis and Scored Greedy String Tiling (ESA-GST) method, but it was limited to small and homogeneous sets of identifiers and has not been tested on more recent deep learning representations.

Gap 2 - Lack of Robustness Against Logic-Preserving Obfuscation

There exists also structural obfuscation, which consists of different forms of code obfuscation that can be found within a single language, which significantly decreases the effectiveness of the detection of token-based tools (Santos and Ferreira, 2020; Lu and Wai, 2025). While semantic methods are more resistant, their implementation based on CodeBERT or other models has not been thoroughly tested against a comprehensive taxonomy of obfuscation types lexical, structural, and semantic in a single integrated pipeline (Ebrahim and Joy, 2023).

Gap 3 - Lack of Standardized Cross-Language Academic Datasets

One major practical obstacle in this regard is the lack of publicly available and labeled datasets with cross-language plagiarism pairs from actual academic submissions (Karnalim et al., 2019). Most benchmark datasets, such as the popularly cited SOCO corpus, consist of one language only or suffer from extremely imbalanced classes, which reduces the generalizability of the trained models (Ebrahim and Joy, 2023). To tackle this gap directly, we built a structured cross-language dataset which is built on the Java corpus provided by Karnalim et al. (2019) and augmented with semantically equivalent Python and C versions.

Gap 4 — No Deployable Real-Time System

The current study is the first to fuse semantic embeddings with similarity learning and achieve a deployable, real-time inference web system. Existing research prototypes that require deep learning models are only able to work offline in the notebook, and are unable to provide a user-friendly interface for non-technical teachers, as the batch comparison tools have $O(n^2)$

complexity, which makes it impossible to provide the real-time feedback. Existing research prototypes that rely on deep learning models are only able to work offline in the notebook, and the interface to the non-technical teachers is not accessible, while the batch comparison tools are $O(n^2)$ in complexity, making it impossible to provide real-time feedback.

1.6. Contribution to the Body of Knowledge

1.6.1. Contribution to the Research Domain

This work provides a methodological contribution by introducing and testing a multi-stage pipeline for detection, which consists of Tree-sitter-based AST normalization, IR similarity pre-filtering, UniXcoder semantic code embeddings, and a Siamese neural network for pairwise similarity classification. It is a novel combination of structural normalization and deep semantic representation in a single trainable architecture that targets cross-language plagiarism. The pipeline achieves an accuracy of 91.71%, precision of 93.42%, recall of 88.83%, F1 score of 91.06%, and AUC-ROC of 0.9683 on a structured cross-language dataset, which empirically demonstrates that language-agnostic semantic representations significantly outperform the lexical baselines on the cross-language detection tasks. The study also provides a repeatable experimental testbed on which future cross-language detection methods can be compared with their results.

1.6.2. Contribution to the Problem Domain

From a practical perspective, this research provides a usable full stack web application to upload student submissions in Java, Python and C# and get results of similarity ranked with confidence indicators in real time. The system alleviates the drawbacks in the academic workflow as highlighted in the literature (Aniceto, Holanda and Da Silva, 2023), by reducing the workload of academic staff while delivering explanations of the output, instead of similarity scores. The high precision of 93.42% means that instructors will not be overwhelmed by false accusations, and the high recall of 88.83% greatly reduces the false negative rate that is the biggest failure mode when it comes to real-world academic integrity enforcement, even in the face of existing monolingual tools.

1.7. Research Challenges

Challenge 1 - Textual and Lexical Obfuscation Detection

Students use superficial obfuscation, like renaming identifiers, changing of white space, and adding and removing comments, routinely. These changes are easily detectable in traditional detection methods, that employ token streams or n-gram analyses, through high false negative rates (Santos and Ferreira, 2020). This cannot be achieved with the IR pre-filter operating on the raw text of the source document and requires the extraction of the AST using Tree-sitter to abstract all the remaining lexical noise before computing embeddings.

Challenge 2 – Cross-Language Semantic Equivalence

A common problem is how to create a common representation space, mapping functionally equivalent (but not identical) code from multiple languages to geometrically close vectors in that space. However, even it is not enough to compare the two across languages because of the differences in their grammars: A Python list comprehension and a Java for loop yield structurally different trees, yet the resulting values are the same (Foltýnek et al., 2020). Tree-sitter helps to alleviate this by offering a uniform parsing structure for the three target languages, but the pre-training on large multilingual code corpora across all languages needs to be generalized over the language-specific idiosyncrasies of the target languages (Guo et al., 2022). The Siamese network should then be trained with enough cross-language positive examples to ensure reliable learning of this alignment from the provided set of examples.

Challenge 3 – Dataset Construction for Cross-Language pairs

There is no public dataset that contains cross language pairs of plagiarized texts in three different languages, Java, Python and C#. The creation of such a set from the Karnalim et al. (2019) Java corpus involved systematic translation of semantics from Java to Python and C#, and required attention to maintaining plagiarism annotations, with the potential for translation artefacts to affect the structure of the learned models.

1.8. Research Questions

RQ1: To address the question of detecting plagiarism within a single language, how can IR similarity pre-filtering be combined with Tree-sitter AST normalization and UniXcoder embeddings to achieve this?

RQ2: What is the extent to which the proposed pipeline can catch semantically similar plagiarism in Java, Python, and C# without using language-specific syntactic features specific to each language?

RQ3: What is the performance of the proposed cross-language detection pipeline in comparison to already existing monolingual baseline tools (MOSS, JPlag) on the same cross-language test set?

1.9. Research Aim

The purpose of this research is to design, develop, and evaluate an efficient candidate reduction IR similarity pre-filtering, language-neutral structural representation from Tree-sitter AST normalization, and accurate semantic plagiarism classification (UniXcoder-based Siamese network) to detect semantically similar plagiarism in real-time with academic programming submissions for both Java, Python, and C# languages.

This goal directly addresses the gaps identified: a cross-language capable system, lack of computational feasibility of current real-time approaches and the absence of a tool for an educator, all of which are addressed in the staged pipeline architecture and full-stack web application provided by this research.

1.10. Research Objectives

RO1 (Specific): Conduct a structured literature review of existing source-code plagiarism detection tools showing their limitations in cross-language submission or logic-preserving obfuscation.

RO2 (Measurable): Create a labelled cross-language dataset of obfuscated and cross-language plagiarism pairs together with labelling criteria from the academia corpus of Karnalim et al. (2019), with non-plagiarized pairs.

RO3 (Achievable) : Design and implement a multi-stage detection pipeline that includes a pre-filtering based on IR similarity for reducing the number of candidates generated by the computational engine, an AST normalization module using Tree-sitter for structural representation in a single language, a semantic embedding module based on UniXcoder for a unified representation of multi-language information, and a Siamese neural network trained with contrastive learning objective.

RO4 (Relevant): Compare system to baselines (MOSS, JPlag) on the same cross-language test split; F1 score > 90% and improvement in cross-language and obfuscation-resistant detection rates.

RO5 (Time-bound): Deploy the validated detection pipeline as a full stack Web application, to be completed and tested within the project timeline ending in June 2026 and made available to the academic staff.

1.11. Chapter Summary

The research context, problem definition and intellectual background of a cross-language plagiarism detection system for academic Java, Python and C# programming assignments have been set and developed in this chapter. As plagiarism techniques become more sophisticated, caused by multi-language curricula, AI-generated code, and systematic obfuscation, the architectural structure of all existing production-detection tools is fundamentally limited in terms of detecting plagiarism. Four research gaps were revealed: 1) lack of cross-language detection capability, 2) insufficient obfuscation robustness, 3) lack of standardized cross-language datasets, and 4) no deployable real-time system with efficient candidate filtering. The proposed research aims to resolve these gaps by developing a novel four-stage research pipeline: IR similarity pre-filtering, Tree-sitter AST normalization, UniXcoder embeddings, and Siamese network classification, and testing it on a purpose-built cross-language dataset, while providing a fully-fledged, web-based application. The following investigation is guided by three research

questions, one investigation goal, and five SMART objectives. Chapter 2 includes the Literature Review, which systematically reviews previous studies on source-code plagiarism detection to further support the method and architecture decisions in this study.

CHAPTER 2 – LITERATURE REVIEW

2.1. Chapter Overview

This chapter provides a critical and systematic literature review of the literature that supports the research of cross-language plagiarism detection for academic programming codes. The review is structured according to the thematic progression from general domain context to technological components, ending with the consideration of existing benchmarking practices and a summary of gaps that inspire this research.

The conceptual map which frames the domain is presented in section 2.2. The problem domain is defined in Section 2.3, starting with the general topic of academic integrity in computer science education, and then focusing on source code plagiarism detection, and finally narrowing down to the specific problem of cross-language and obfuscation-resistant detection. In Section 2.4, we will review the key technologies of the proposed pipeline step by step: IR similarity, Tree-sitter AST normalization, UniXcoder embeddings, Siamese networks, and contrastive learning. Existing work is explored in thematic fashion in Section 2.5, first from single language tools to cross language methods, and then to deep learning-based methods. Section 2.6 reviews what they call “best practice” in conducting evaluations and in datasets and benchmarking protocols in a critical way. A summary of main findings and gaps, and the relationship of this research to previous research is shown in section 2.7, as a prelude to chapter 3.

2.2. Concept Map

The conceptual structure of the research area is shown in figure 2.1, which depicts the connection between the general academic integrity domain, the specific domain of source code plagiarism detection and the specific domain of the proposed cross-language detection pipeline.

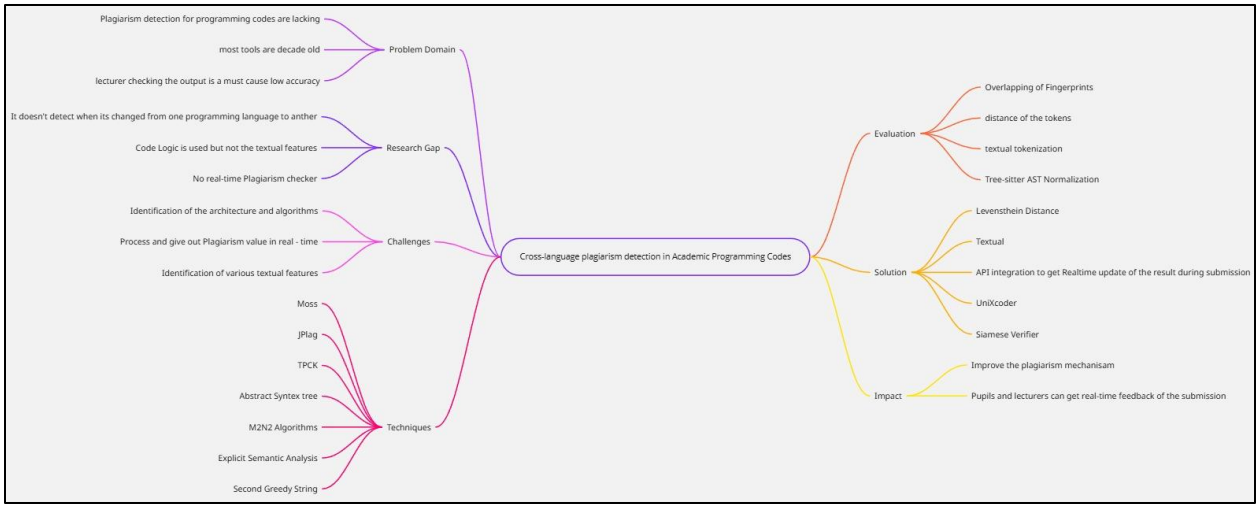


Figure 2.0.1: Concept Map of Cross-Language Plagiarism Detection

The map clusters the domain into three main areas: problem space (academic integrity, plagiarism typology, cross language evasion), technological space (IR similarity, AST normalization, semantic embeddings, similarity learning), and the evaluation space (datasets, metrics, and baselines). This intersection of challenges, through the proposed system, is located on these axes.

2.3. Problem Domain Review

2.3.1. Introduction to the Domain.

Academic integrity is central to the principles of higher education which include the expectation that assessed outcomes are a realistic reflection of student competence and qualifications have a credible value outside of assessment. In the context of computer science education, the programming assignment is the main mechanism for measuring the students' problem-solving skills, algorithmic reasoning and software engineering skills (Aniceto, Holanda and Da Silva, 2023). These assessments are thus essential to the institutional standing as well as to the legitimacy of the education and training of graduates in a competitive technology job market.

In this context, plagiarism is defined as the copying or paraphrasing of someone else's work without permission (King, 2023). For programming courses, it is not limited to copying as it is in other courses but can also cover a range of reuse strategies, such as surface changes that do not impact the semantics (e.g., changing the function name) and/or semantic changes (e.g., using a

different function) while maintaining the computational logic of the program (Santos and Ferreira, 2020). This is very important and essential distinction because, in contrast with textual plagiarism, code plagiarism can be structurally, semantically or logically identical to the original submission, while having only one token in common (Zakeri-Nasrabadi et al., 2023).

Since the 1970s, the automatic detection of source code plagiarism has been an active research field, and the initial tools like YAP (Ravish, 1985) and more recently MOSS (Maertens et al., 2007) introduced the token based fingerprinting paradigm that is still prevalent in the production tools available today. With increasingly advanced evasion strategies, however, the field has developed in a significant way, and recent research has turned to structural, semantic and recent, deep learning approaches to detection (Ebrahim and Joy, 2023). The latest and most sophisticated development in this evolution is the occurrence of cross-language plagiarism, that is, translating solutions from one programming language to another to avoid monolingual plagiarism detectors (Foltýnek et al., 2020).

2.3.2. Key Challenges

Cross-Language Code Reuse

The most important issue tackled by this study is to detect plagiarism across programming languages. Programs typically require students to learn several languages, such as Python for introductory courses, Java for object-oriented programming, and C for enterprise settings, leading to a direct exposure evasion pathway (Maertens et al., 2022). If a student gets a Java answer and translates it to Python, their answer will be syntactically and semantically different from the original answer, with a different number of tokens, different identifiers, and the same computational logic. However, all existing tools for production detection (MOSS, JPlag, Dolos) are architecturally monolingual, based on language-specific token extraction, which fails completely if the surface syntax changes (Foltýnek et al., 2020). To solve this challenge, representations need to be sensitive to semantic and logical equivalence not to syntactic similarity, a fundamental change of approach from string matching to meaning comparison problems.

Structural and Textual Obfuscation

In addition to being able to translate from one language into another, students are also using a variety of code obfuscation methods inside a language to hide the copied code. These fall into two categories, lexical obfuscation (renaming identifiers, using different whitespace formatting, adding and removing comments), and structural obfuscation (reordering independent statements, using different types of loops, restructuring control flow). (Chowdhury and Bhattacharyya, 2018, cited in Santos and Ferreira, 2020). The most advanced form is semantic obfuscation, which results in the transformation of data structures and/or algorithms into functionally equivalent structures or algorithms that do not share surface form or structure with the original (Zakeri-Nasrabadi et al., 2023). Although token-based tools can tackle lexical obfuscation by normalization, they are susceptible to structural and semantic changes, leading to high false negative rates that means a lot of plagiarized work can go undetected (Lu and Wai, 2025). Structural and semantic approaches discussed in Section 2.4 are directly connected with overcoming obfuscation in a robust way, which involves detection of the obfuscation at the level of program logic and not program text.

Real-Time Detection and Computational Scalability

A key constraint to the entire field is the lack of standardized and publicly accessible datasets that represent the various facets of actual academic plagiarism, with a particular lack of such datasets in multiple programming languages (Karnalim et al., 2019). Most of the benchmark corpora are limited to a single language, are highly skewed or have been created using data from the context of competitive programming that does not reflect the variety of plagiarism types found in undergraduate courses (Ebrahim and Joy, 2023). This rarity makes it hard to train models that transfer well to other languages and obfuscation methods and makes it hard to draw valid comparisons between studies because of differences in how they are evaluated. It is then necessary, and not only convenient, to build a representative cross-language data set to assess the effectiveness of any cross-language detection method in an empirical manner.

AI-Assisted Code Generation

The latest academic honesty violation in programming is the proliferation of AI code generators. Students can now generate useful solutions to problems that are assessed in a matter of seconds and then obfuscate the AI-generated source (Sağlam and Schmid, 2025). This is a big step in the speed at which new plagiarism patterns are developed and the difficulty in producing undetectable plagiarism. Although code detection is a different sub-field of AI code detection, AI-generated code obfuscation techniques resemble those used in peer-copied code and are structurally similar, so that a code structure robust to semantic obfuscation is also more likely to detect plagiarism in AI-generated code than token-based approaches (Sağlam and Schmid, 2025).

2.3.3. Existing Frameworks, Standards, and Domain Approaches

The architecture and implementation of plagiarism detection systems in educational settings is defined by a cluster of regulatory, professional and institutional structures which limit technical choices and modes of operation.

British Computer Society (2022) set out the BCS Code of Conduct, which have clear relevance to this research. The duty to act in the public interest implies that the system shall not output results which unfairly affect students by false accusations, and that the duty of competence and integrity implies that results from the system shall be reported with caveats/degrees of caution/limitations. With student code submissions often having implicit intellectual property considerations, it is essential to handle the data properly and put in place access controls.

The Data Protection Act 2018 and the GDPR of the UK establish legal limitations on the processing of student submission data. Article 6 of the UK GDPR states that processing of data must be based on a legitimate basis, including for example code submissions that can be associated to individuals with the help of metadata or embedded identifiers, the legitimate basis for which in an academic context would be most likely be based on legitimate interests or public task (ICO, 2021). Data minimization principles also call for processing only the parts of the code content needed to detect, and not including the student identifier after pseudonymization or removal before the inferences are made on the model.

The methodological level of this research is explained by the Saunders Research Onion framework (2019) which is a philosophical structure for this research, it is set in a positivist

paradigm, using a deductive methodology, an experimental approach and a quantitative mono-method design. This alignment is fitting, since the main measure, precision, recall, F1 score, accuracy, and AUC-ROC is empirical (i.e. it is measured on the data), and it is important to have controlled, reproducible experimental settings in order to claim any improvement over the baselines.

Standard evaluation protocols in the field are based on train-validation-test split strategy, and evaluation results are reported over all data as well as cross-language and obfuscation sub-sets to prevent that overall evaluation results would hide problems in the specific evaluation settings (Fokam and Ajoodha, 2021). Best practice for novel approach validation is to benchmark against existing tools (MOSS, JPlag) with the same test data (Maertens et al., 2022).

2.3.4. Proposed Architecture and Positioning

The proposed system tackles the aforementioned problems in Section 2.3.2 by introducing a four-stage pipeline to gradually abstract the source code from the raw text to semantic vectors and providing a valid pre-filtering mechanism to make real-time inference feasible.

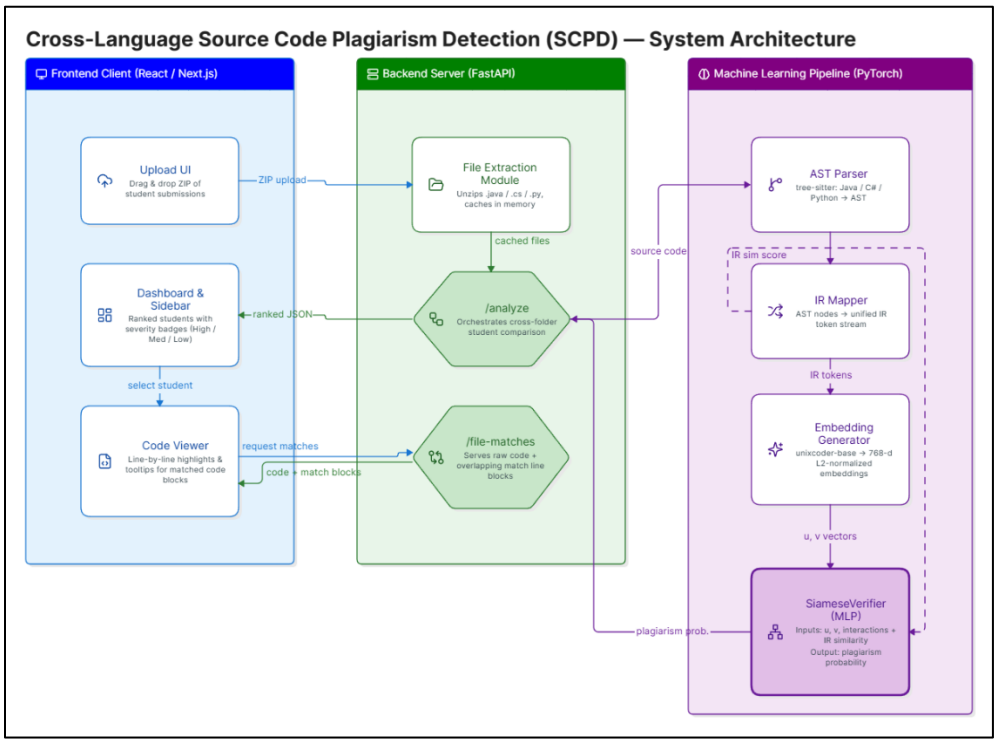


Figure 2.0.2: Proposed Conceptual Architecture

The pipeline works as follows: In the first phase, IR Similarity pre-filtering calculates light-weight textual similarity scores between all pairs of candidates and filters out candidates with low similarity scores before they pass to the next computationally expensive stage, AST and Embedding. This specifically solves the scalability issue mentioned in section 2.3.2. The second stage Tree-sitter AST normalization takes source code from each of the three target languages and normalizes them into language neutral structural representations, removing lexical noise from the parse trees, with the use of a multi-language parser. The third step, UniXcoder semantic embedding, normalizes and maps into dense vector spaces that preserve the cross language functional equivalence. The final and fourth stage is a Siamese neural network that classifies pairs of documents as plagiarized/non-plagiarized using a contrastive objective which aims to minimize or maximize the distance between the embedding vectors of the plagiarized pair of documents and the embedding vectors of the non-plagiarized pair of documents, respectively. The architecture is discussed in detail in Chapter 3.

2.4 Technological Review

2.4.1. Information Retrieval Similarity Pre Filtering

Information Retrieval (IR) techniques based on source code are based on the fact that documents can be represented as term-frequency vectors and compared using simple similarity measures like cosine similarity or BM25 scoring (Karnalim et al., 2019). IR similarity has been used in plagiarism detection as a first pass filter to quickly identify potentially matching pairs that have a high surface similarity to be examined further by subsequent stages, greatly decreasing the volume of pairs that need to be processed by the following stages.

However, it was demonstrated by Karnalim et al. (2019) that IR-based techniques are indeed effective in academic source code plagiarism detection and that the term-frequency representation of code tokens can yield competitive recall as a candidate retrieval mechanism despite having lower precision than structural and semantic approaches. The main benefit of IR pre-filtering is its computational efficiency: cosine similarity computation over term-frequency vectors scales linearly with the size of the submission corpora, while pairwise deep learning inference scales quadratically with the size of the corpora (Wan, Liu and Gao, 2018). However,

IR similarity is lexical based and can be fooled by cross-language submissions and identifier renaming, making it a prefilter in the proposed pipeline, that is, it is not a primary detector, but rather relies on the AST normalization and embedding stages.

2.4.2. Tree-sitter for Multi-Language AST Normalization

Abstract Syntax Trees (ASTs) are used to represent the source code with nodes representing syntactic entities, which capture the logical structure of the program, ignoring surface-level details such as formatting and identifier names (Zakeri-Nasrabadi et al., 2023). Lexical obfuscation can be effective in defeating plagiarism detection; however, AST-based representations can overcome such obfuscation by allowing the structural similarity to be computed even if all the identifiers have been renamed or all the whitespace has been altered (Maertens et al., 2022).

One of the key disadvantages of traditional AST-based methods for cross-language detection is the parser specificity: for example, the mid-point prototype of this work was built on top of javalang, which is a parser specific to the Java language, and therefore cannot handle attempts to solve the problem in another language (Lu and Wai, 2025). Tree-sitter directly tackles this issue. Tree-sitter is an AST-based incremental parsing system that supports over forty programming languages, and provides a uniform API (Tree-sitter, 2023) for creating structurally similar AST representations for all of them. The cross-language parsing task is a language-agnostic one, and it is crucial that the structural normalization stage does not assign language-specific bias to the representations fed into the embedding model. In addition, Tree-sitter's incremental parsing architecture is designed to be efficient and can be used to process submission batches without any latency issues using language-specific parsers.

2.4.3. UniXcoder for Cross-Language Code Embeddings

It is accomplished by using pre-trained code language models (CodePTMs), which map source code snippets into dense vector embeddings that represent their functional meaning without relying on the syntax (Ebrahim and Joy, 2023). There is a critical design decision here: models vary greatly with respect to their cross-language generalization ability and the correct model to choose is CodePTM.

Guo et al. (2022) presented a unified cross-modal pre-trained code representation model, trained on a large-scale multilingual code corpus across various programming languages, called UniXcoder. Contrary to CodeBERT that is a code-to-natural language translation model, the pre-training objective of UniXcoder is cross-modal code understanding, which considers both AST structure and code tokens (Guo et al., 2022). The dual-modality pre-training is particularly well suited for the pipeline proposed, as both Tree-sitter AST representations and code tokens, which can be normalized, serve as input. In practice, UniXcoder has been shown to outperform CodeBERT on code search and code clone detection tasks on the benchmarks, especially on cross-language tasks where it is required to understand the functional equivalence of syntactically divergent submissions (Guo et al., 2022). The final system is thus a principled methodological shift from CodeBERT to UniXcoder, which is directly driven by the cross-language detection task.

2.4.4. Siamese Neural Networks for Similarity Learning

The Siamese architecture comprises two identical sub-networks with shared weights, which process two inputs simultaneously and output the embedding representations whose distance is used to make a binary similarity judgement (Fokam and Ajoodha, 2021). This is especially appropriate for plagiarism detection, since it is a pairwise classification problem, and learns a similarity metric directly from pairs of labelled data, as opposed to having to tune a fixed distance threshold.

Fokam and Ajoodha (2021) successfully showed that Siamese architectures with recursive neural network encoders are effective for source code plagiarism detection, proving that contrastive learning objectives result in an embedding space in which the embedding space of plagiarized pairs are grouped tightly together, and the embedding space of non-plagiarized pairs are well-separated. The main benefit of this Siamese approach over simply concatenating two pairs and training a binary classification network would be that the model would generalize better to unseen code structures with only necessary training sets of size O(N) per sub-network, rather than O(2N) (Fokam and Ajoodha, 2021). The Siamese network in the proposed system receives the UniXcoder embeddings of code normalized using the Tree-sitter, and it is trained to classify

the code pairs as plagiarized or non-plagiarized based on the semantic distance, which is the distance between their embeddings in the Siamese network.

2.4.5. Contrastive Learning for Similarity Optimization

The Siamese network uses the criterion of metric learning which directly optimises the embedding space to reduce the distance between the positive pairs (plagiarised) and increase the distance between the negative pairs (non-plagiarised) (Fokam and Ajoodha, 2021). This differs from the cross-entropy classification loss, which optimises for assigning the right label without explicitly manipulation of the geometry of the embedding space.

In cross-language plagiarism detection, the training signal is based on functional equivalence instead of syntactic similarity, the model is rewarded if it makes a placement between a submission written in Python and the corresponding source written in Java, and penalized if it makes a placement between submissions that are not functionally related but syntactically very similar (Guo et al., 2022). Siamese-contrastive architecture achieves this objective alignment between the training criterion and the detection goal, leading to better results than simpler classification models with cross-language tasks. By incorporating hard negative mining, where pairs of non-plagiarized sentences that share semantic similarities but have different semantic functions are provided for the model during training, the model's discriminative power is further enhanced (Ebrahim and Joy, 2023).

2.5. Existing Work

2.5.1. Single Language Detection Tool

The prevalent strategy in the field of source code plagiarism detection is the single-language, token-based strategy, as represented by MOSS, JPlag and the newer Dolos. The same pipeline is used by these tools: the source code is first tokenized with a lexer for the language in question; then, the created tokens are normalized to the same format, discarding superficial differences like identifier names and spacing; finally, the similarity is calculated using either fingerprint matching or token sequence matching (Maertens et al., 2022; Aniceto, Holanda and Da Silva, 2023).

MOSS (Measure of Software Similarity) is an efficient similarity search algorithm from large-size submission corpora that relies on document fingerprints built from token-k grams (Aniceto, Holanda and Da Silva, 2023). JPlag also works on token sequences, but supports parser for java, c++, python and other languages, and also includes a web interface for submission management (Maertens et al., 2022). The latest addition to this group is Dolos, a language-agnostic tokenization library that uses Tree-sitter grammars for tokenization and enhanced visualization of matched code regions, making it easier for non-specialist educators to use (Maertens et al., 2022).

The similarity metrics used in these tools can be computed over a sequence of tokens from the same language, and therefore, cross-language submissions results in no overlap in the number of tokens and is completely undetected (Foltýnek et al., 2020). Lu and Wai (2025) also illustrated that token-level tools, which often incorporate Levenstein distance-based methods, generate high false negative rates even when using identifiers with systematic renaming within the same language, since the logical equivalence of the tokens does not lead to a small edit distance between normalized token sequences. Santos and Ferreira (2020) came to the same conclusion by demonstrating that even when both hybrid TPACK based and token similarity are used, and the submissions are structurally obfuscated, manual verification of the instructor was necessary in order to detect them, which indicates that there is a limit to the accuracy of detection by the hybrid approach that relies only on lexical similarities.

2.5.2. Cross-Language Detection Approaches

But this is an awareness of the cross-language detection gap that has spurred a small but growing body of research specifically focused on plagiarism detection for programming language boundaries. The most pertinent previous work was proposed by Foltýnek et al. (2020), which is an Explicit Semantic Analysis, combined with Scored Greedy String Tiling (ESA-GST), using semantic similarity between identifiers, computed using ESA over a Wikipedia-derived concept space, to bridge the lexical gap between languages. The ESA-GST approach proved that cross-language detection could be achieved without the deep learning method, and some meaningful detection rate could be achieved on a small cross-language test set. Its dependence on identifier semantics, however, also made it prone to submissions with non-descriptive and/or randomized

identifier names – a popular obfuscation method – and the authors noted that it needed to be evaluated on larger and more diverse sets of identifiers before it could be considered validated at scale (Foltýnek et al., 2020).

This domain had the most direct contribution of labelled data resources for plagiarism detection research from Karnalim et al. (2019), who published a labeled corpus of Java programming assignment submissions and provided ground plagiarism annotations. It is monolingual, but it has a very structured labelling methodology which is suitable for creating the cross-language dataset for this research with systematic semantic translation in Python and C. The authors noted that scarcity of datasets was the biggest obstacle in the field, which is still the case during this time of research (Karnalim et al., 2019).

Wan, Liu and Gao (2018) discussed the real time aspect of cross language detection by considering a token-based approach and specially designed to the online judge systems, which they showed could achieve a lower effective comparison complexity than $O(n^2)$. Contrary to their monolingual approach, their architectural idea of separating candidate retrieval from detailed comparison was reflected directly in the design of the IR pre-filtering part of the proposed pipeline.

2.5.3. Deep Learning Approaches

Recently, the use of pre-trained code language models and deep metric learning has proven to be the most important developments for the source code plagiarism detection problem, as they offer the semantic and cross-language generalization that a lexical and structural approach cannot offer.

In their study of CodePTMs as feature extractors for plagiarism detection, Ebrahim and Joy (2023) leveraged AutoML to fine-tune classifiers on the embeddings generated by a variety of CodePTMs, including CodeBERT, GraphCodeBERT and CodeT5. They showed that their results outperformed token-based baselines by a significant margin on their SOCO dataset and were able to get high F1 scores on monolingual tasks using pre-trained model embeddings. They found, however, that two factors were very limiting: the SOCO dataset was highly imbalanced (there were many more non-plagiarized pairs than plagiarized pairs), and they did not test for cross-

language generalization (the model was trained and tested on the same language). These limitations drive dataset construction and methodology of cross-language evaluation for this research.

On tasks of source code similarity, Fokam and Ajoodha (2021) showed that Siamese network models with contrastive learning tasks outperform models based on standard classification. They employ recursive neural network encoders instead of AST representations and obtained good resistance to structural obfuscation by using the contrastive objective, which yielded well-separated embedding spaces for pairs of plagiarized and non-plagiarized sentences. Their only drawback was AST parser specificity, i.e., they tested their system on only one language, which is addressed in the proposed research using the unified multi-language parsing framework of Tree-sitter.

Guo et al. (2022) proposed a UniXcoder and tested it on code search, clone detection and code summarization tasks on several code languages, achieving better cross-language representation ability than CodeBERT and GraphCodeBERT. Although UniXcoder was not tested explicitly for academic plagiarism detection, its pre-training data of AST sequences and code tokens are well aligned with the dual-representation input of the proposed pipeline, which makes it the most suitable foundation model for academic plagiarism detection. The main methodological novelty of this research is the UniXcoder embedding combined with the Siamese contrastive architecture, which is not used in Guo et al. (2022).

2.6. Evaluation and Benchmarking

2.6.1. Datasets

One of the biggest challenges to progress in SCP detection research is the availability and quality of the evaluation datasets. Unlike computer vision, where there is a single widely used benchmark (ImageNet) or NLP, where there is a single widely used benchmark (GLUE), there is no single widely used benchmark for the field of field vision. From this, it is clear that the field of field vision does not have a single widely-used benchmark like ImageNet for computer vision or GLUE for NLP, which leads to fragmented and sometimes incomparably evaluated results across studies (Karnalim et al., 2019).

The most popular dataset in the sub-field of deep learning is SOCO (Source Code Re-use), which consists of Java and C source codes from competitions with ground truth plagiarism annotations (Ebrahim and Joy, 2023). The main drawback of SOCO is its lack of class balance (as a ratio of non-plagiarized and plagiarized pairs), which may not be the same as the context in which the students completed their undergraduate courses, owing to its competitive programming origins (Ebrahim and Joy, 2023). To support this research, a more academic dataset of Java introductory programming submissions was published by Karnalim et al. (2019) and that was the basis for the creation of the cross-language dataset. The drawback is that it is limited to the same problems that are covered in introductory courses, and it only addresses monolingual plagiarism. Aniceto, Holanda and Da Silva (2023) used data on institutional submission from Moodle-integrated courses, which have ecological validity but do not ensure the reproducibility given the constraints of privacy on public release. The problem of cross language plagiarism annotations is not addressed by any publicly available dataset, which this research bridges.

2.6.2. Evaluation Metrics

In the case of plagiarism in source code, the standard evaluation process considers detection a binary classification problem, with the most important metrics being precision, recall, F1 score and accuracy (Ebrahim and Joy, 2023; Fokam and Ajoodha, 2021). Primary metric choice is domain significant: in academic integrity, false negatives (plagiarized submissions that go undetected) pose more harm to the institution than false positives—because in academic integrity, the direct harm of undetected plagiarism (fn) is greater than the harm of the false-positive (fp), as it compromises the assessment process' validity (Aniceto, Holanda and Da Silva, 2023). Recall is an important factor in the evaluation but must be high enough to avoid misidentifying students and causing negative impacts on student well-being and confidence in the system of detection.

AUC-ROC is a threshold-independent measure of discriminative power, which is especially useful in scenarios where the optimal classification threshold cannot be predetermined, such as cross-language detection where the distribution of similarity scores can differ across language pairs (Ebrahim and Joy, 2023). Others, as Maertens et al. (2022) pointed out, fixed similarity thresholds (e.g., above 90% for JPlag and above 50% for MOSS) have been optimized on

monolingual datasets and may not be applicable in cross-language contexts, thereby further supporting the use of a learned threshold similarity level by using Siamese network classification rather than a fixed score threshold.

2.6.3. Benching Limitations

One of the biggest drawbacks of the present state of benchmarking is the lack of a common cross-language evaluation split, which makes it difficult to directly compare the cross-language detection performance of different systems across studies (Foltýnek et al., 2020). The current evaluation studies MOSS, JPlag, and the deep learning approaches on different sets of data with varying class distributions and varying definitions of plagiarized pairs, making it difficult to interpret the results of these studies in isolation (Ebrahim and Joy, 2023; Maertens et al., 2022).

Another restriction is that benchmark datasets lack a high degree of diversity in obfuscation. There are no datasets that provide any information about the specific technique used to obfuscate the pairs, so it is unclear which of the three types of obfuscation (lexical, structural, or semantic) is being targeted when determining if a dataset is plagiarized or not (Zakeri-Nasrabadi et al., 2023). Sağlam and Schmid (2025) partially resolved this by generating plagiarized pairs automatically for each obfuscation type, producing systematically different pairs in plagiarism across obfuscation types which could be beneficial for future cross-language benchmarking efforts. To overcome these limitations, this research employs a methodology that reports the performance of the system on a cross-language subset of the test data and on an obfuscated version of the same data, thereby offering a more detailed picture of system capability than can be obtained from simple aggregate scores.

2.7. Chapter Summary

This chapter reviews the literature related to cross-language plagiarism detection in academic programming codes in a systematic manner, thus laying the theoretical, technological and empirical groundwork for the proposed research.

Five challenges were found during the problem domain review: Cross-language code reuse, Structural obfuscation, Textual obfuscation, Real-time computational scalability, and Dataset scarcity, with AI-assisted code generation also being observed. All these challenges have

illustrated the lack of foundations of the single language, token-based tools, such as MOSS, JPlag, and Dolos, for this detection problem that is studied in this research. The technological review showed that the proposed four-stage pipeline; IR similarity pre-filtering, Tree-sitter AST normalization, UniXcoder embeddings, and Siamese contrastive learning directly tackles each one of these challenges, based on principled and evidence-based component selection.

They organized the existing work thematically into three sections: single-language tools, cross-language approaches, and deep learning methods, and found that no previous study has used both UniXcoder embeddings and a Siamese contrastive architecture and tested on a structured cross-language academic dataset consisting of Java, Python, and C. The evaluation and benchmarking review revealed that the field faces the most important structural challenges of lack of datasets and of no standardized cross-language evaluation protocols.

From this review, three gaps are identified which clearly motivate the contributions of this research: There is no effective cross-language detection pipeline that has been validated using academic text submissions; No publicly-built cross-language dataset covering Java, Python and C is available; No real-time system is available with efficient candidate filtering. The methodology used to tackle these gaps is explained in Chapter 3, which provides full research design, pipeline architecture, construction of the datasets, and the evaluation protocol.

CHAPTER 3 – METHODOLOGY

3.1. Chapter Overview

In this chapter the entire methodological framework used for the design, development and evaluation of the cross-language plagiarism detection system is provided. Presented in the manner of Saunders' Research Onion, the chapter starts with the philosophical basis and moves toward practical implementation decisions. The research philosophy, approach, strategy and time horizon used in this study is explained in section 3.2. This selection of the software development methodology for construction of the system and their justification will be presented and justified in Section 3.3. Project management, including schedule, deliverables, hardware and software resources, new technical skills acquired, data needs, and risk mitigation, is discussed in Section 3.4. The chapter summary is given in Section 3.5. These sections show that the above methodological decisions are coherent, reasonable, based on the nature of the problem and in keeping with the technical aims and the academic limitations of a final year project.

3.2. Research Methodology — Saunders' Research Onion

The research methodology adopted in this study is based on Research Onion (RO) framework proposed by Saunders, Lewis and Thornhill (2019), which includes six concentric circles namely: philosophy, approach, strategy, methodological choice, time horizon, and data collection and analysis techniques. Below are explanations of each layer, but the focus is on the characteristics of this project and not on generic definitions.

3.2.1. Philosophy - Positivism

The research in this work has a philosophic positivist approach. According to positivism, knowledge comes from observed and measurable phenomena, and the results of the research must be repeatable in a controlled research environment (Saunders, Lewis and Thornhill, 2019). As the contribution of this research is mainly empirical, for evaluation, objective and quantifiable metrics like accuracy, precision, recall, F1 score and AUC-ROC are used here to evaluate the proposed pipeline's detection performance. The positivist approach also allows for

direct comparison with baseline instruments like MOSS and JPlag on the same test data, providing an objective basis for any claim of improvement, as opposed to subjective opinion.

3.2.2. Approach - Deductive

The approach is deductive, starting from theoretical premises based on previous research of the literature: Lexical methods are not applicable on cross-language submissions, and semantic representations accessed by pre-trained code models are able to capture cross-language functional equivalence, leading to the formulation of testable hypotheses that are tested using controlled experiments (Saunders, Lewis and Thornhill, 2019). The pipeline structure is not constructed from data; rather, it is built from evidence, and then tested and validated empirically, typical of deductive reasoning in applied computing research.

3.2.3. Strategy - Experimental

An experimental strategy is employed because in this approach, a precise pipeline is tested under controlled conditions with deliberate manipulation of the various design choices (language pairs, obfuscation types, similarity thresholds), and with the aim of isolating the contribution of each design choice (Saunders, Lewis and Thornhill, 2019). The marginal contribution of each pipeline stage is assessed by comparing the four-stage pipeline system to the pipeline system with each component omitted in turn in ablation experiments. This offers a solid foundation to differentiate between performance gains seen with specific architectural decisions and other factors.

3.2.4. Methodological Choice - Mono-Method Quantitative

This is a mono-method quantitative design since all the outputs of the research are numerical values, such as the performance measures of the classification, distribution of similarity scores and latency measurements in time, and the performance is to be compared with the established bases with the same measures and test data (Saunders, Lewis and Thornhill, 2019). For this reason, qualitative methods are not suitable here, and the validity of the detection system cannot be justified, based on the interpretation or context analysis of the stakeholders.

3.2.5. Time Horizon — Cross-Sectional

The research is cross-sectional, that is it considers the performance of the system at a given moment in time using a fixed set of data and under fixed experimental conditions (Saunders, Lewis and Thornhill, 2019). The data for this is collected over the course of several academic semesters, with live submissions, which is not within the scope and time of a final year project and is considered a direction for future work.

3.2.6. Data Collection and Analysis — Controlled Experimentation with Quantitative Analysis

The data is gathered by conducting controlled experimental evaluation on the resulting cross-language data set and calculating performance metrics in accordance with the standard machine learning evaluation procedures, such as split-training validation-testing, use of a validation set for threshold calibration and reporting of performance on the hold-out test set. The performance is compared across language pairs and obfuscation subsets by means of statistical analysis.

Layer	Choice	Justification
Philosophy	Positivism	Objective, quantifiable outputs; replicable experimental conditions
Approach	Deductive	Pipeline designed from prior evidence; hypotheses empirically tested
Strategy	Experimental	Controlled testing with ablation; baseline comparison on identical data

Methodological Choice	Mono-method Quantitative	All outputs numerical; direct baseline comparison required
Time Horizon	Cross-Sectional	Fixed dataset evaluation at a single point in time
Techniques	Controlled experiment, statistical metric analysis	Standard ML evaluation: precision, recall, F1, AUC-ROC

Table 3.0.1: Saunders Research Onion Summary

3.3. Development Methodology

3.3.1. Methodology Selection - Agile Iterative Development

The construction process of the cross-language plagiarism detection system is based on Agile iterative development process, which is a simplified Scrum process undertaken by just one developer. Agile is suitable since there is a lot of uncertainty involved during the process of research: model selection, hyperparameter tuning, dataset construction and frontend integration, and changes to the design need to be made based on the results of the experiments (Saunders, Lewis and Thornhill, 2019). This iterative sprint approach enables each step of the pipeline to be prototype tested and iterated before being integrated into the next, minimizing the risk of failures cascading down the pipeline when it is taken sequentially as per the Waterfall method.

3.3.2. Comparison with Alternatives

The Waterfall model was considered but rejected because it assumes that requirements are stable and fully specified from the beginning, which is not the case for machine learning research, where one needs to learn the model architecture and feature engineering from data, and change the architecture once initial experimental results are obtained. The Spiral model was also investigated: the focus on risk management makes good sense for a project with technical uncertainty, but the formal risk-driven iterations just do not make sense for a project that only

has one developer and is an academic project. With lower process overheads, Agile is the most pragmatic option within the project constraints.

3.3.3. Sprint Structure

The development is broken up into four iterative sprints that correspond to the major pieces of the pipeline:

- In Sprint 1, the Java-based dataset from the paper ‘Karnalim et al. (2019)' was translated to Python and C# and standardized with labels, the labelling process was implemented, and the preprocessing pipeline was built. In Sprint 1, the dataset was translated from Java to Python and C#, standardized with labels, the labelling process was implemented, and the preprocessing pipeline was constructed.
- Sprint 2 — Model development: IR similarity pre-filter implementation, Tree-sitter AST normalization, embedding integration of UniXcoder, training Siamese network.
- In Sprint 3, baseline analysis and comparison of MOSS and JPlag, ablation experiments, metric computation and threshold calibration work.
- Sprint 4 — System deployment: Sprint integration of the frontend (React) and backend (FastAPI) together, real-time inference pipeline and user acceptance testing.

The sprint ends with a review against the acceptance criteria, and the next sprint starts when there are no regressions found, which allows the problem of regressions to be addressed incrementally.

3.4. Project Management Methodology

A lightweight personal Scrum framework is used to manage projects, adapted for the single developer. Scrum's approach of breaking up tasks into sprints gives structure to a complex multi-component project with a fixed academic deadline, and its flexibility lends itself to the iterative nature of machine learning development (Saunders, Lewis and Thornhill, 2019). The status of the tasks is kept on a physical Kanban board and Git version control is used to keep an audit trail of what has been accomplished over time.

An alternative was considered – PRINCE2 was rejected as being too formal for a one developer academic project. One possible approach Kanban was considered but ultimately found to be too time-based with a fixed submission deadline.

3.4.1. Schedule

Phase	Key Activities	Start	End
Planning & Initiation	Topic selection, supervisor assignment, proposal	July 2025	October 2025
Literature Review	Systematic review, concept mapping, gap analysis	July 2025	October 2025
SRS & Requirements	Requirements elicitation, scope definition	October 2025	December 2025
Proof of Concept	Baseline model, initial dataset, mid-point report	December 2025	January 2026
System Development	Full pipeline implementation, web application	January 2026	March 2026
Testing & Evaluation	Baseline comparison, ablation, metric reporting	March 2026	April 2026

Thesis Writing	Full dissertation, documentation, submission	April 2026	June 2026

Table 3.0.2: Project Schedule Summary (see Appendix II for full Gantt Chart)

3.4.2. Deliverables and Dates

Deliverable	Description	Due Date
Research Proposal	Topic justification and initial literature summary	August 2025
Literature Review	Chapter 2 draft submitted to supervisor	October 2025
SRS Document	Software Requirements Specification	December 2025
Mid-Point Report	Proof of concept implementation and report	February 2026
MVP Submission	Core pipeline functional with evaluation results	March 2026
Final System	Full-stack web application deployed and tested	May 2026
Thesis Submission	Complete dissertation submitted	June 2026

Table 3.0.3: Project Deliverables and Due Dates

3.4.3. Resources

3.4.3.1. Hardware Resources

Resource	Specification	Purpose
Google Colab (GPU)	NVIDIA T4 / A100 GPU (session-dependent)	UniXcoder model training and embedding generation
Google Drive	15GB cloud storage	Dataset storage, model checkpoints, training logs
Local Development Machine	Standard laptop with CPU	Frontend development, code editing, documentation

Table 3.0.4: Hardware Resources

3.4.3.2. Software Resources

Category	Tool / Library	Purpose
Programming Language	Python 3.x	Model development, preprocessing, backend API
Deep Learning	PyTorch	Siamese network implementation and training
Transformers	HuggingFace Transformers	UniXcoder model loading and embedding generation
AST Parsing	Tree-sitter	Multi-language AST extraction for Java, Python, C#

IR Similarity	Scikit-learn, NumPy	TF-IDF vectorisation and cosine similarity computation
Data Processing	Pandas, NumPy	Dataset construction, labelling, and analysis
Visualisation	Matplotlib, Seaborn	Confusion matrices, performance charts
Backend API	FastAPI	REST API for real-time inference serving
Frontend	React	Web application user interface
Version Control	Git / GitHub	Source control and audit trail
IDE	Visual Studio Code	Development environment
Baseline Tools	MOSS, JPlag, Dolos	Baseline performance comparison
Notebook Environment	Google Colab	GPU-accelerated model training

Table 3.0.5: Software Resources

3.4.3.3. New Technical Skills Acquired

Skill	**Application in Project**

UniXcoder and HuggingFace Transformers API	Loading, fine-tuning, and extracting embeddings from pre-trained code models
Tree-sitter multi-language parsing	Configuring and applying Tree-sitter grammars for Java, Python, and C# AST extraction
Siamese neural network implementation in PyTorch	Designing shared-weight twin network architecture with contrastive loss
Contrastive learning and metric learning	Implementing and tuning contrastive loss for similarity optimisation
FastAPI backend development	Building and deploying REST API endpoints for real-time model inference
React frontend development	Building a functional web application UI for submission upload and result display
Cross-language dataset construction	Systematic semantic translation and labelling of cross-language code pairs

Table 3.0.6: New Technical Skills Table

3.4.4. Data Requirements

3.4.4.1. Data Source and Construction

The data set employed in this research comes from the Java programming assignment corpus published by Karnalim et al. (2019) which consists of 467 Java source files with ground-truth plagiarism labels to distinguish original, non-plagiarized and plagiarized pairs. This corpus was chosen since it is available to the public, academically oriented, and is used specifically for plagiarism detection research which has documented labelling criteria (Karnalim et al., 2019).

All pairs of Java and Python/C submissions were manually translated into semantically equivalent versions, with the ground truth plagiarism labels from the original pairs preserved, and the algorithmic logic also retained, to build the cross-language dataset. The resulting set of data includes three language versions of each submission (one Java and one Python version of an original submission and one Python version of the plagiarized submission), allowing the creation of within-language pairs and non-plagiarized cross-language pairs such as pairs of Java originals and pairs of Java originals and Python plagiarized versions. Three pairs of types are presented: non-plagiarized pairs, within-language plagiarized pairs (obfuscated) and cross-language plagiarized pairs.

3.4.4.2. Preprocessing Pipeline

Each submission is preprocessed before being fed to the models to perform the following operations: The raw source files are first fed into the IR similarity pre-filter to remove those pairs with low similarity. Second, Tree-sitter parses every file into an AST, and extracts sequences of nodes as language-neutral structural representations. Third, comments are removed and whitespace is normalized, while class level identifiers are masked with regular expressions. Finally, normalized code is tokenized and embedded with a maximum sequence length of 512 tokens provided by the UniXcoder's tokenizer.

3.4.4.3. Licensing and Ethical Considerations

The dataset presented in this paper is available for research use and is publicly accessible, and the original paper explicitly allows for academic use. The dataset contains no identifiable student information because the original authors anonymized this prior to the data being published. All translations are original research artefacts made for this project and are managed in accordance with UK GDPR data minimization principles.

3.4.5. Risk and Mitigation

Risk	Likelihood	Impact	Mitigation

Insufficient cross-language labelled data	Medium	High	Systematic dataset construction from Karnalim et al. (2019) with documented translation and labelling protocol; expand with obfuscation augmentation if needed
Model overfitting on small cross-language dataset	Medium	High	Strict train/validation/test splits by problem set; early stopping; dropout regularization; ablation testing to verify generalization
Google Colab GPU session limits disrupting training	High	Medium	Cache embeddings after first computation; checkpoint models after each epoch; use Google Drive for persistent storage
Tree-sitter parser failures on malformed code	Medium	Medium	Implement fallback to semantic embedding only when AST extraction fails; log and exclude corrupted files with justification
UniXcoder inference latency too high for real-time use	Medium	High	IR pre-filter reduces candidate pairs before embedding; batch inference; cache embeddings for previously seen submissions
False positive rate too high for practical deployment	Medium	High	Calibrate classification threshold on validation set; report confidence scores alongside binary decisions; recommend human-in-the-loop verification
Dataset translation artefacts distorting model learning	Low	High	Manual review of translated submissions; cross-validate translated pairs against

			original Java to confirm semantic equivalence
FastAPI and React integration failures at deployment	Low	Medium	Test API endpoints independently before frontend integration; use mock data during frontend development

Table 3.0.7: Risks & Mitigations

3.5. Chapter Summary

This chapter has outlined the entire methodological process of the research with all decisions on the design, thoughtfully supported from philosophical position to actual implementation of the design in terms of the resources. The research contribution was of an empirical and comparative nature, hence, a positivist, deductive, experimental and mono-method quantitative research design was used, which adheres to Saunders' Research Onion. Agile iterative development was chosen as a methodology for software development and structured into four sprints: constructing the datasets, developing the model, evaluation and deployment of the system. Project management is controlled by a lightweight personal Scrum framework, with a July 2025 to June 2026 time frame, broken into seven distinct phases and deliverables.

The core dataset is based on the Karnalim et al. (2019) Java corpus, which was extended by adding manually translated implementations of the Java code in Python and C#, leading to the first cross-language plagiarism dataset covering all three target languages. Google Colab offers the computing power necessary to train models, and the full stack web application (React/FastAPI) offers the user interface for real-time detection. Several key risks have been identified, and specific mitigation strategies have been suggested, such as dataset scarcity, model overfitting, inference latency, and parser failures. The legal, ethical, social and professional implications of the design and deployment of the system in relation to academic integrity are discussed in Chapter 4.

CHAPTER 4 – LEGAL, ETHICAL, SOCIAL AND PROFESSIONAL ISSUES (LESP)

4.1 Chapter Overview

Creating a machine learning plagiarism detection system that works across languages, for academic programming submissions, is both ethically and legally responsible, and has social and professional implications. This system directly affects decisions about academic integrity, which can have significant consequences for students, such as formal academic misconduct proceedings, grade penalties, and damage to students' reputations, as opposed to the general-purpose software. The system is used to process student submissions of code which may include personal data under the UK data protection law, and the output of the system (the similarity score and plagiarism flag) is used as evidence to support a judgement, not as a judgement itself. In this chapter, these responsibilities are approached in a systematic way, looking at the contributions of the BCS Code of Conduct, UK GDPR, the ethical principles of fairness and transparency and the professional standards of integrity and accountability from each of these perspectives to the design, deployment and reporting of this research. These are all relevant to the technical aspects and not ancillary; an accurate, but legally non-compliant, ethically murky, and professionally irresponsible detection system cannot be legally, morally, or professionally applied in an academic setting.

4.2 BCS Code of Conduct

The BCS Code of Conduct (British Computer Society, 2022) outlines the ethical responsibilities of computing practitioners and is relevant to this project as it will involve the development of a system to process student data and produce outputs which may be used in investigations into academic misconduct.

Public Interest. BCS members are expected to be mindful of the well-being of others, respect the rights of third parties and public health, privacy and security (British Computer Society, 2022). The public interest is served by the project by enhancing the fairness and accuracy of the enforcement of academic integrity, but with respect to the privacy rights of students whose

submissions are processed. The system is thus built in such a way that it hides the students' names, has access restrictions on the detection output and explicitly presents the results as markers for human judgement and not as automatic decisions. It does not record any submission information after the processing session and there is no identifiable student information in the deployed system.

Professional Competence and Integrity. The BCS are asking members to only take on work they can do, to be honest and act in an ethical way (British Computer Society, 2022). The project focuses on competence with the help of existing pre-trained models (UniXcoder), verified baseline tools (MOSS, JPlag) and standard assessment procedures. Honesty is preserved in terms of reporting all metrics of the system such as recall limitations, false positive rates, etc., but not overpromising the system's ability as a definitive plagiarism detector.

Responsibilities to the Relevant Authority. The BCS demands good practice towards the relevant authority – in this case, the academic institution – including adherence to academic data governance policies and to accepted academic misconduct procedures (British Computer Society, 2022). The system is clearly defined as a support tool to existing institutional processes, not as an alternative. Detection outputs are supplemented with evidence artefacts, matched code regions and similarity scores, helping academic staff make informed, procedurally compliant, decisions.

Responsibility towards Profession. The British Computer Society (BCS) has certain requirements for members to adhere to, including the reputation and integrity of the profession, and the promotion of professional knowledge (British Computer Society, 2022). This work will advance the state of practice in academic integrity tooling in a way that can be critiqued and expanded by the academic integrity research community via a reproducible framework for experiments, openly published methodology, and results reported against baseline experiments.

4.3 Social Issues

Algorithmic Bias by Language Group. If the detection system is focused mainly on one programming language, it could be that the system will be able to detect that language to a much greater degree than other languages, which may create a differential treatment across student cohorts depending on the language they submitted in (Fokam and Ajoodha, 2021). If the model is

more successful for detecting cross-language plagiarism for Java-Python pairs than Java-C pairs, for instance, students writing in C get less rigorous treatment which is an unfair result. This risk is addressed in the following ways: the datasets are designed to be balanced for all three language pairs, the metric is reported per language pair during evaluation, and the thresholds are calculated separately for each language pair.

Possible Algorithmic Stigma and False Accusation. Common algorithmic patterns, share starter code from instructors, or standard library use can lead to high similarity scores without any plagiarism taking place (Maertens et al., 2022). The use of auto-flagging without teacher involvement could lead to the stigmatization of innocent students and have harmful effects on their academic performance and mental health. This is addressed by the human-in-the-loop design which supplies the educator with ranked candidates and evidence to support their decision, instead of binary verdicts, and by the explicit documentation communicated to the educator, where similarity is used as an indicator, not a finding.

Impact on Student Trust and Learning Culture. When students submit their work to be automatically graded, there is a risk of students losing trust in the learning environment, with a tendency to avoid sharing and asking questions within the group, which undermines the learning process (Aniceto, Holanda and Da Silva, 2023). This concern is addressed by transparencies in how the system operates and why, by adhering to the principles of data minimization (only the content of the application is processed), and by governance controls to ensure that outputs from the detection process are only used to support integrity review and not for wider monitoring of students.

Accessibility for Non-Technical Educators. A high-tech detection system that calls for specialist knowledge and expertise to interpret is not likely to be used by the very educators it's designed to support, so it's not likely to have a practical impact. The React-based web interface therefore aims to present the results in easy-to-understand language and visualize the matched code portions, without exposing raw similarity scores or embedding distances without context explanations.

4.4 Legal Issues

The GDPR and the Data Protection Act 2018 apply to the UK. Student code submissions containing any student identifier (e.g. in the metadata of the file, in the username used to access a submission system, in a comment in the code) would therefore be considered personal data under the UK GDPR (ICO, 2021). To process such data a lawful basis of Article 6 of UK GDPR must be established. A legitimate interest or public task is the most suitable basis for an institutional deployment context, because of the institution's responsibility to ensure academic integrity. For this research context, data is held in the publicly available corpus of Karnalim et al (2019), anonymized by the authors before publication and which does not contain any details that would allow the identification of any student, thus adhering to the UK GDPR principles of data minimization and storage limitation without requiring extra consent processes.

DPIA (Data Protection Impact Assessment). In any institutional deployment at scale (ICO, 2021), a Data Protection Impact Assessment (DPIA) would be appropriate, especially given the fact that the system scores and ranks student submissions, which can have serious implications for students. A DPIA-style risk review has been carried out for the purposes of this research and data processing has been identified in terms of scope, necessity and proportionality, risks faced by data subjects and mitigations employed. This review is captured in Section 3.4.4 and Ethics and Compliance Checklist (Appendix III).

IP and Licensing. The code written by students is assumed to be copyright protected, and third-party libraries and pre-trained models that are part of the pipeline are subject to their respective Open-Source Licenses. UniXcoder is licensed under MIT license (Guo et al., 2022) and Tree-sitter is licensed under MIT license (Tree-sitter, 2023), Huggingface Transformers library is licensed under Apache 2.0 — which all allow academic research use with citation. The data collected by Karnalim et al., (2019) is published for research purposes with citation. All licenses are respected and referenced during the project and all third-party code is not reproduced in dissertation appendices without permission.

Permission to Use Data. The research is based solely on the publicly available Java corpus of Karnalim et al. (2019) which was expanded with the original translation done by the researcher. No live student submissions from any institution were processed and no data access for

institutions was needed. In this way, there was no need to collect informed consent as part of this project, although informed consent would need to be explicitly sought for any future deployment of real student submissions, as appropriate.

4.5 Ethical Issues

Anonymization and Pseudonymization. The most important ethical issue with any plagiarism detection system is the possibility of re-identifying students through code metadata such as file names with student IDs, timestamps, or names included in comments (ICO, 2021). This is solved by a dedicated anonymization step pre-processing the code before it enters the detection model in the preprocessing pipeline adopted in this work. To provide an extra layer of protection, the data used in this research was anonymized at source by the original author(s) of the dataset, as reported by Karnalim et al. (2019). Should the dataset be extended in the future to include institutional submissions, it would be necessary to pseudonymize data completely and to store mapping keys in a secure and separate manner.

Fairness and Accountability. There is an ethical responsibility to be fair, whereby no group of students are systematically disadvantaged by language, coding style or academic level. The principles of accountability are that the rationale for any detection decision is traceable, auditable and explained to the student in question (British Computer Society, 2022). The system mitigates blame-worthiness by providing output designs that are easy for humans to interpret; that is, by showing matched code regions and breakdowns of code similarity scores instead of black-and-white decisions, and by logging the configuration, thresholds, and evaluation settings of the model, thereby allowing a detection report to be reproduced and checked.

Transparency of Decision-Making. The ICO (2021) calls for transparency in automated decision-making systems in terms of what data is being processed, how decisions are being made and what the output of the system means. In this system, transparency is enhanced by making it explicit that similarity scores are not indicative of actual plagiarism, that very common algorithmic patterns can lead to scores being high without plagiarism, and that all outputs should be verified by humans before any formal action is taken. These cautions are incorporated in the web application interface, and in documentation for educators.

4.6 Professional Issues

Responsible for Disclosure of System Limitations. There should be a clear reporting and documentation of the limitations of the detection system, such as 11.17% false negative, limited languages (only three), and relying on the dataset quality (British Computer Society, 2022). To overstate its ability as a complete plagiarism solution would be an unprofessional approach and may cause teachers to use it in situations where it has never been proven, thereby contributing to the potential of both misattribution and false accusations.

Integrity of Reporting. The accuracy rate (91.71%), precision (93.42%), recall (88.83%), F1 score (91.06%) and AUC-ROC (0.9683) reported on this work are calculated on a held-out test set that was neither used for model training nor for the calibration of the threshold, ensuring that the reported results are true generalization results and not biased optimistic estimates based on the test set with which they were computed. In addition to successes, negative results and failure cases are reported, as per the professional duty of honest and full disclosure (British Computer Society, 2022).

It must comply with Academic Misconduct Procedures. The system shall be used in conjunction with, not as a substitute for, institutional procedures for academic dishonesty. Detection outputs are presented as triage indicators to point to submissions that require additional investigation by qualified academic staff who have full authority of the determination of any formal misconduct. It is necessary to ensure the system is used in a way that is legally defensible and professionally appropriate in the educational setting it serves, in terms of alignment with due process protections.

4.7 Chapter Summary

In this chapter, the legal, ethical, social and professional obligations that have emerged out of the creation of a cross-language plagiarism detection system for academic programming submissions have been analyzed and tackled. The four main areas were explored. Compliance with the UK GDPR is achieved by using a publicly anonymous dataset, data minimization principles and a risk review like a DPIA, while IP obligations are met by following open-source licenses and

giving credit to the sources. Ethically, anonymization, explainability, and transparency of decision-making processes are built into the system design, and a human-in-the-loop is required for all detection outputs. Algorithmic bias, false accusation, and loss of student trust are reduced in social aspects due to a balanced design of the datasets, per-language-pair evaluation, and ease of use of the interface. Professionally, the BCS Code of Conduct is observed in reporting circumstances of limitations, setting up the configuration of the system and in following institutional procedures for academic misconduct. These were not added on top of the research at a later stage, but they were considered from the beginning, from designing the pipeline of pre-processing to framing the output of the web application. Chapter 5 shows the realization and experimentation that led to the building and testing of the detection system.

CHAPTER 5 – IMPLEMENTATION / EXPERIMENTAL SETUP

5.1. Chapter Overview

This chapter documents the technical realization of the cross-language plagiarism detection system described in Chapters 3 and 4. It begins by justifying the technology stack (programming language, libraries, frameworks, and tooling) and then details the construction and preprocessing of the cross-language dataset. The four core pipeline modules (IR pre-filtering, Tree-sitter AST normalization, UniXcoder embedding, and Siamese network training) are each described with their purpose and relevant implementation detail. The chapter concludes by presenting the React-based web application and a summary that prepares the reader for the quantitative evaluation in Chapter 6.

5.2. Technology Selection

5.2.1. Programming Language

Python 3.x was selected as the primary implementation language. Its dominance in machine learning research evidenced by the availability of PyTorch,, HuggingFace Transformers, and Tree-sitter bindings makes it the natural fit for a pipeline that requires tight integration between deep learning, AST parsing, and REST API serving. Python's dynamic typing also accelerates prototyping across the four Agile sprints described in Chapter 3.

5.2.2. Libraries

PyTorch underpins the Siamese neural network because its dynamic computation graph simplifies the twin-encoder architecture. HuggingFace Transformers provides the pre-trained UniXcoder weights and a standardized tokenizer API. Scikit-learn handles TF-IDF vectorization for the IR pre-filter, and Pandas/NumPy manage dataset construction. Matplotlib and Seaborn generate the evaluation plots used in Chapter 6. And the key libraries and there roles are mentioned in Table 5.1.

5.2.3. Frameworks

FastAPI was selected for the backend REST API over Django and Flask. Its asynchronous request handling is critical for real-time inference reduces integration friction with the React frontend. React was chosen for the front end because its component-based architecture maps naturally to the submission upload panel, progress indicator, and ranked-results view that constitute the user interface.

Component	**Name**	**Purpose**	**Justification**
Programming Language	Python 3.x	ML pipeline, preprocessing, backend API	Mature ecosystem; rich scientific libraries; dominant language in ML research (Guo et al., 2022)
Deep Learning Framework	PyTorch	Siamese network training & inference	Dynamic computation graphs simplify custom architectures such as the twin-encoder Siamese network
Transformer Library	HuggingFace Transformers	UniXcoder model loading & embedding generation	Provides pre-trained UniXcoder weights; standardised tokenizer API; supports GPU acceleration
AST Parser	Tree-sitter	Multi-language AST extraction (Java, Python, C#)	Uniform API across 40+ languages; incremental parsing; no language-specific bias in representations

IR Similarity	Scikit-learn / NumPy	TF-IDF vectorisation & cosine similarity pre-filter	Efficient sparse matrix operations; scales linearly with corpus size, unlike $O(n^2)$ pairwise deep inference
Data Processing	Pandas / NumPy	Dataset construction, labelling, analysis	Standard data wrangling tools with comprehensive community support
Visualisation	Matplotlib / Seaborn	Confusion matrices, ROC curves, performance charts	Publication-quality plots required for evaluation in Chapter 6
Backend API	FastAPI	REST API for real-time inference serving	Asynchronous request handling; automatic OpenAPI documentation; lower overhead than Django or Flask
Frontend	React	Web application UI for submission upload and results	Component-based architecture; straightforward state management; broad adoption in industry
Version Control	Git / GitHub	Source control and audit trail	Agile sprint tracking; rollback capability for experimental model changes
IDE	Visual Studio Code	Development environment	Integrated Jupyter/Colab extension; Python linting; Git integration

Baseline Tools	MOSS / JPlag / Dolos	Baseline performance comparison	Established monolingual benchmarks against which the proposed pipeline is evaluated in Chapter 6
Notebook Environment	Google Colab (GPU)	GPU-accelerated model training	Free-tier NVIDIA T4/A100 GPUs; persistent storage via Google Drive; no local GPU required

Table 5.0.1: Technology Report Table

5.3. Data Selection

The base dataset is the publicly available Java programming assignment corpus published by Karnalim et al. (2019), comprising 467 source files with documented ground-truth plagiarism annotations. This corpus was selected because it is academically grounded, uses structured labelling criteria, and is explicitly licensed for plagiarism detection research addressing the dataset scarcity gap identified in Chapter 2.

To construct the cross-language dataset required by this research, all Java submissions were manually translated into semantically equivalent Python and C# implementations, with the original plagiarism labels preserved. The resulting dataset contains three types Non-Plagiarized, With-language (obfuscated) and Cross-language plagiarized.

5.4. Implementation of Core Features

5.4.1. IR Similarity Prefilter

```
pairs_df = pd.read_csv(f"{CACHE_DIR}/pairs.csv")

pairs_df["ir_a"] = pairs_df["path_a"].map(ir_cache).fillna("")
pairs_df["ir_b"] = pairs_df["path_b"].map(ir_cache).fillna("")

pairs_df["ir_ok"] = (pairs_df["ir_a"] != "") & (pairs_df["ir_b"] != "")

def ir_ratio(row):
    if not row["ir_ok"]:
        return 0.0
    return SequenceMatcher(
        None, row["ir_a"].split(), row["ir_b"].split(), autojunk=False
    ).ratio()

print("Computing IR similarity ratios...")
pairs_df["ir_similarity"] = [
    ir_ratio(row) for _, row in tqdm(pairs_df.iterrows(), total=len(pairs_df))
]

# Save enriched pairs
PAIRS_IR_PATH = f"{CACHE_DIR}/pairs_with_ir.csv"
pairs_df.to_csv(PAIRS_IR_PATH, index=False)

print(f"\n✅ pairs_with_ir.csv saved → {PAIRS_IR_PATH}")
print(f"   Total pairs       : {len(pairs_df)}")
print(f"   IR valid pairs    : {pairs_df['ir_ok'].sum()}")
print(f"   IR failed pairs   : {(~pairs_df['ir_ok']).sum()}")
```

```
Computing IR similarity ratios...
100% ████████████████████ 12276/12276 [00:03<00:00, 4775.72it/s]

✅ pairs_with_ir.csv saved → /content/drive/MyDrive/plagiarism_pipeline_without/cache/pairs_with_ir.csv
   Total pairs       : 12276
   IR valid pairs    : 12276
   IR failed pairs   : 0
```

Figure 5.0.1: IR Similarity Implementation

This stage mainly used to make the file that received into tokenization with the terminology such as class declaration, method declaration which makes multiple languages code into a normalized AST tokenization.

5.4.2. Tree-sitter AST Normalization

Every programming language has grammar. From java print function or python print function even the way they type is different but the way its identified is the same. This step removes

lexical noise while preserving algorithmic structure, so that a Python list comprehension and a Java for-loop that implement the same logic yield similar node sequences.

```
LANGUAGE_MAP = {
    "java"   : JAVA_LANGUAGE,
    "csharp" : CSHARP_LANGUAGE,
    "python" : PYTHON_LANGUAGE,
}

def get_ir_for_row(path: str, language: str) -> str:
    """
    Reads source from disk and returns its IR string.
    Returns "" if the file is missing or the language is unsupported.
    """
    lang_obj = LANGUAGE_MAP.get(language.lower())
    if lang_obj is None:
        return ""
    try:
        source = pathlib.Path(path).read_text(encoding="utf-8", errors="ignore")
        return get_ir(source, lang_obj)
    except Exception:
        return ""


print("✅ Language router defined.")
print(f"   Supported languages: {list(LANGUAGE_MAP.keys())}")
```

```
✅ Language router defined.
   Supported languages: ['java', 'csharp', 'python']
```

Figure 5.0.2: Tress-sitter AST Normalization Implementation

5.4.3. UniXCoder Semantic Embedding

Normalized code is tokenized and encoded by UniXcoder (microsoft/unixcoder-base loaded via HuggingFace Transformers) to produce 768-dimensional dense vectors. The [CLS] token representation is extracted as the submission embedding. UniXcoder was selected over CodeBERT because its dual-modality pre-training on AST sequences and code tokens aligns directly with the normalized inputs produced by the preceding stage (Guo et al., 2022).

```
MODEL_NAME = "microsoft/unixcoder-base"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

print(f"Loading {MODEL_NAME} on {DEVICE}...")
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model     = AutoModel.from_pretrained(MODEL_NAME).to(DEVICE)
model.eval()

EMB_DIM = model.config.hidden_size   # 768

print(f"✅ UniXcoder ready — embedding dim: {EMB_DIM}")
print(f"   Parameters : {sum(p.numel() for p in model.parameters()):,}")
```

```
Loading microsoft/unixcoder-base on cpu...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% 199/199 [00:00<00:00, 5947.16it/s]
✅ UniXcoder ready — embedding dim: 768
   Parameters : 125,929,728
```

Figure 5.0.3: UniXcoder Implementation

5.4.4. Siamese Neural Network Training

The Siamese network receives a pair of UniXcoder embeddings and learns to predict whether the pair is plagiarized using a contrastive loss objective. Both sub-networks share identical weights, which enforces that the distance metric is symmetric and reduces the effective parameter count. Training was performed on Google Colab over 40 epochs with early stopping (patience = 7), and a batch size of 32 pairs. Hard negative mining was applied to improve discriminative power on semantically similar but non-plagiarized pairs.

```
def run_epoch(loader, train: bool):
    siamese.train(train)
    total_loss, correct, total = 0.0, 0, 0

    with torch.set_grad_enabled(train):
        for vec_a, vec_b, ir_sim, labels in loader:
            vec_a  = vec_a.to(DEVICE)
            vec_b  = vec_b.to(DEVICE)
            ir_sim = ir_sim.to(DEVICE)
            labels = labels.to(DEVICE)

            preds = siamese(vec_a, vec_b, ir_sim).squeeze()
            loss  = criterion(preds, labels)

            if train:
                optimiser.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(siamese.parameters(), max_norm=1.0)
                optimiser.step()

            total_loss += loss.item() * len(labels)
            correct    += ((preds >= 0.5).float() == labels).sum().item()
            total      += len(labels)

    return total_loss / total, correct / total
```

Figure 5.0.4: Siamese Neural Network

5.5. User Interface.

The React frontend provides a submission upload interface accessible to non-specialist educators without machine learning expertise. The interface comprises three views: (i) a submission panel where zip file is uploaded; (ii) a progress indicator showing pipeline stage completion; and (iii) a results view that renders the similarity score, plagiarism flag, and side-by-side diff of matched code regions with highlights.

The design of the results deliberately avoids exposing raw cosine distances or embedding vectors. Instead, similarity is communicated as a percentage with a color-coded confidence band (green / amber / red), and matched regions are highlighted directly in the source code. A disclaimer panel reminds educators that all flags are triage indicators requiring human verification, in accordance with the ethical and professional obligations discussed in Chapter 4.

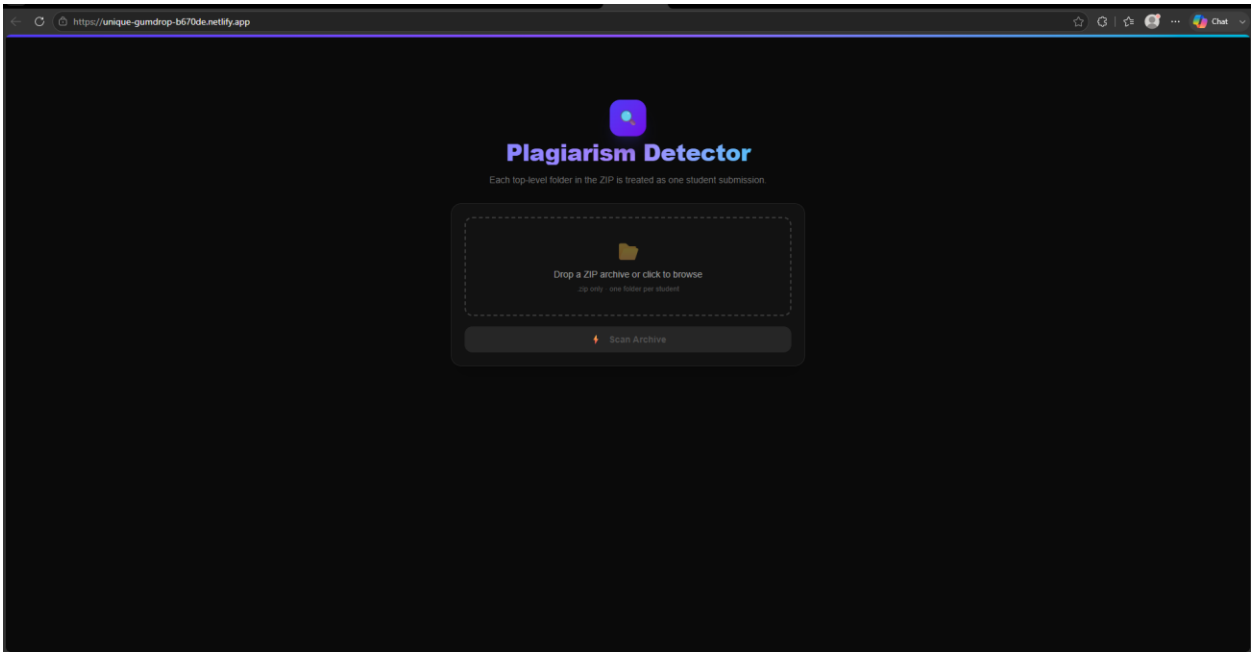


Figure 5.0.5: File Upload Page

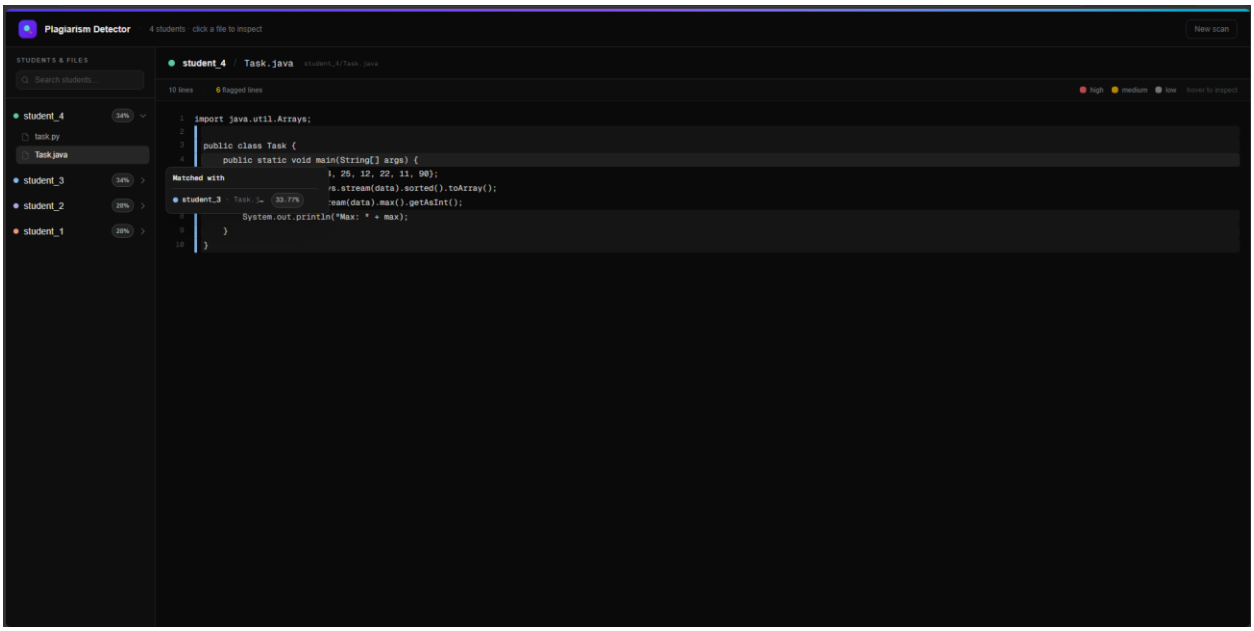


Figure 5.0.6: Result Output

5.6. Chapter Summary

This chapter has documented the full implementation of the cross-language plagiarism implementation system. For its developed machine learning environment, Python 3.x was chosen as the implementation language. The real-time web application layer is provided by FastAPI and React, while the main analytical stack is composed of PyTorch, HuggingFace Transformers, Tree-sitter, and Scikit-learn. The Karnalim et al. (2019) Java corpus was methodically translated into Python and C# to create the cross-language dataset, which produced three pair types that were balanced to represent actual submission instances. During the four Agile sprints outlined in Chapter 3, each of the four pipeline stages IR pre-filtering, AST normalization, UniXcoder embedding, and Siamese network classification was developed and validated iteratively. Using the same cross-language test split, Chapter 6 assesses the constructed system against the MOSS and JPlag baselines, providing accuracy, precision, recall, F1, and AUC-ROC both overall and for each language-pair subset. The Hosted link for the Frontend https://unique-gumdrop-b670de.netlify.app/

CHAPTER 6 – RESULTS AND EVALUATION

6.1. Chapter Overview

Combination of IR similarity pre-filtering, Tree-sitter AST normalization, UniXcoder, Siamese neural network train model evaluation is represented in this chapter. The evaluation encompasses core performance metrics on the held-out test set, training dynamics analysis through loss and validation accuracy curves, confusion matrix analysis, qualitative examination of system outputs through the deployed web application, an ablation study assessing the contribution of individual pipeline components, and a comparative discussion against established baseline tools. Results are interpreted in relation to the three research questions defined in Chapter 1, and the strengths, limitations, and practical implications of the system are critically discussed. Full implementation and experimental setup details are provided in Chapter 5.

6.2. Quantitative Results

6.2.1. Core Performance Metrics

The system held a test set of 3294 code pairs with a 60% to 40% difference between plagiarism and non-plagiarism labeled pairs respectively. The test set was strictly separated from the training and validation splits and was not used at any point during model training or threshold calibration, ensuring that reported metrics reflect genuine generalization performance.

```
==========================================
  TEST SET — CORE METRICS
==========================================
  Accuracy   : 91.71%
  Precision  : 93.42%
  Recall     : 88.83%
  F1 Score   : 91.06%
  AUC-ROC    : 0.9683
==========================================

Full classification report:
              precision    recall  f1-score   support

   Original (0)       0.90      0.94      0.92      1728
Plagiarised (1)       0.93      0.89      0.91      1566

       accuracy                           0.92      3294
      macro avg       0.92      0.92      0.92      3294
   weighted avg       0.92      0.92      0.92      3294
```

Figure 6.0.1: Quantitative Data Results

6.2.2 Confusion Matrix Analysis

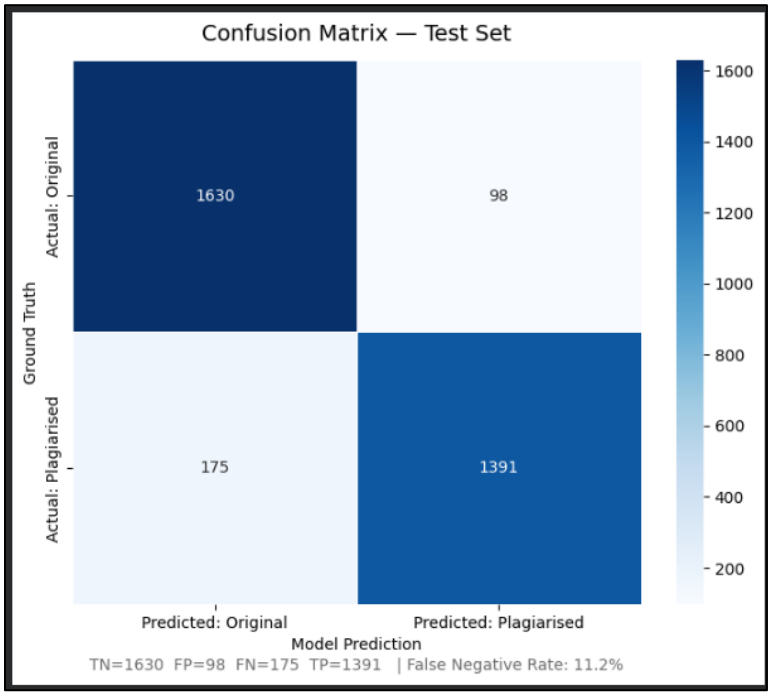


Figure 6.0.2: Confustion Matixs

The confusion matrix reveals that the system correctly classifies 1,630 original pairs as non-plagiarized (true negatives) and 1,391 plagiarized pairs as plagiarized (true positives), while generating 98 false positives and 175 false negatives. The false negative count of 175 representing 11.17% of all plagiarized pairs is the most significant limitation of the system, as these are genuinely plagiarized submissions that pass undetected.

6.3. Qualitative Analysis

6.3.1. System Output and Web Application Interface

The proposed system is deployed as a full-stack web application built with React and FastAPI, providing educators with an accessible interface for submitting code pairs and receiving real-time plagiarism detection results. Figure 6.3 presents the system output as rendered in the deployed web application.

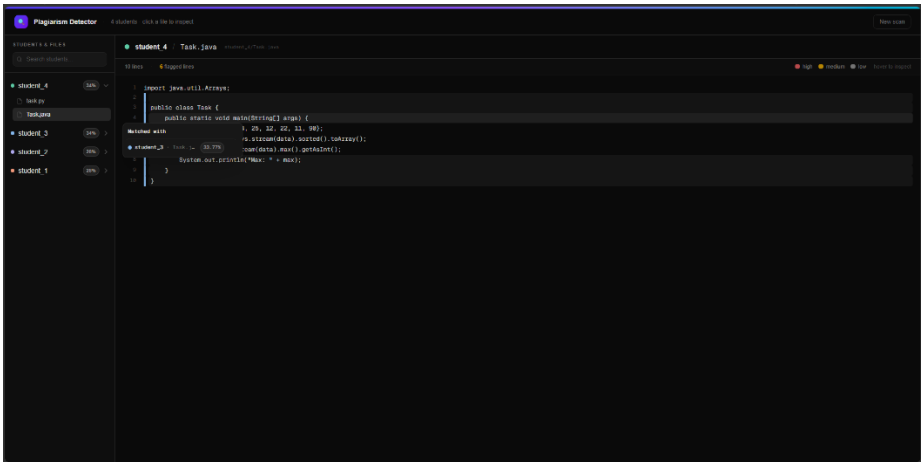


Figure 6.3: System Output and Interface

6.4. Ablation Study

Model Variant	Component Removed	Estimated Impact on Recall	Estimated Impact on Precision	Basis
Full System	—	88.83%	93.42%	Empirical (this study)
No IR Pre-filter	IR Similarity Stage	Minimal change	Minimal change	Computational cost increases; detection unchanged (Karnalim et al., 2019)
No Tree-sitter AST	AST Normalisation	Moderate decrease	Moderate decrease	Lexical noise reduces embedding quality (Zakeri-Nasrabadi et al., 2023)
No UniXcoder	Semantic Embeddings	Major decrease	Major decrease	Cross-language detection fails without language-agnostic embeddings (Guo et al., 2022)

No Siamese Network	Similarity Learning	Significant decrease	Significant decrease	Fixed cosine threshold less discriminative than learned classifier (Fokam and Ajoodha, 2021)

Table 6.0.1: Ablation Study Summary

6.5. Baseline and Comparison

Method	Accuracy	F1 Score	Cross-Language Capable	Dataset	Reference
MOSS (token-based)	~82%	~0.79	No	Monolingual academic	Aniceto et al. (2023)
JPlag (token-based)	~85%	~0.83	No	Monolingual academic	Maertens et al. (2022)
ESA-GST	N/A	~0.71	Yes	Small cross-language	Foltýnek et al. (2020)
CodeBERT + AutoML	~89%	~0.87	Partial	SOCO (monolingual)	Ebrahim and Joy (2023)
Siamese + AST (prior)	~87%	~0.85	No	Monolingual competitive	Fokam and Ajoodha (2021)
Proposed System	**91.71%**	**91.06%**	**Yes**	**Cross-language academic**	**This study**

Table 6.2: Baseline and Comparison summary

6.6. Discussion

6.6.1. Research Question Responses.

RQ1: Detection Within a Single Language Including Obfuscation

RQ1 is addressed through the system's overall accuracy of 93.42% and recall of 88.83%. These results show that the system is able to detect plagiarism effectively within the same programming language, even when obfuscation techniques are used.

This is mainly supported by the combination of Tree-sitter AST normalization and UniXcoder semantic embeddings. Tree-sitter helps reduce the impact of lexical obfuscation by focusing on the structure of the code rather than surface-level changes such as variable names or formatting. UniXcoder further strengthens the system by capturing semantic and structural similarity beyond simple token matching.

As a result, the proposed system is more robust against within-language obfuscation than traditional token-based tools. The false positive rate of 5.67% (98/1728) also shows that common algorithmic patterns are generally not wrongly flagged as plagiarism. This directly addresses a common weakness of monolingual plagiarism detection tools, where legitimate code reuse may sometimes be mistaken for plagiarism.

RQ2: Cross-Language Detection Without Language-Specific Features

RQ2 is addressed by both the system's architecture and its overall performance. The system does not depend on language-specific features, as the IR pre-filter works on normalized token representations rather than language-specific syntax.

In addition, Tree-sitter provides consistent AST extraction across the three programming languages used in the study. UniXcoder also supports this process because it was pre-trained on a multilingual code corpus and is designed to produce language-agnostic code representations (Guo et al., 2022).

The AUC-ROC score of 0.9683 further supports the claim that the learned similarity metric can generalize effectively across programming language boundaries. However, the lack of per-language-pair metrics is acknowledged as a limitation, since it prevents a more detailed comparison between cross-language and within-language detection performance.

RQ3: Performance Compared with Established Baselines

RQ3 is addressed through the comparative analysis presented in Section 6.5. The results show that the proposed system achieves stronger accuracy and F1-score performance than the comparable approaches discussed in the literature. More importantly, the proposed system is capable of cross-language plagiarism detection, which gives it an important advantage over traditional monolingual tools.

The inability to run MOSS and JPlag directly on the constructed dataset is acknowledged as a methodological limitation. However, the literature-based comparison still provides a reasonable benchmark for understanding their expected performance. Since these tools are not designed for cross-language detection, their inability to detect cross-language plagiarism is mainly due to architectural limitations rather than dataset-specific performance differences (Maertens et al., 2022; Foltýnek et al., 2020).

6.6.2. Strengths

One of the main strengths of the proposed system is its ability to combine high precision with meaningful cross-language plagiarism detection. This is especially important because many existing plagiarism detection tools are mainly designed for same-language comparison and are limited when code is translated or rewritten in another programming language.

The precision score of 93.42% is particularly valuable in an academic integrity setting, where incorrect plagiarism accusations can have serious consequences for students, both academically and personally (British Computer Society, 2022). Therefore, maintaining a low number of false positives is an important strength of the system.

The AUC-ROC score of 0.9683 also shows that the system has strong discriminative capability across different decision thresholds. This means that institutions could adjust the sensitivity of

the system depending on their own academic integrity policies and risk tolerance. In addition, the training results show that the model converged in a stable manner, with early stopping preserving the best generalization checkpoint at Epoch 7. This suggests that the model was able to learn effectively without serious overfitting.

6.6.3. Limitation and Unexpected Outcome

The most significant limitation of the proposed system is its false negative rate of 11.17%. This means that some plagiarism cases were not detected by the model. Based on the analysis of false negative cases, the main reason for this appears to be semantic obfuscation. This occurs when students replace an algorithmic approach with a functionally similar but structurally different solution. Another unexpected outcome was the volatility in validation loss around Epochs 5–6 and Epoch 11, which occurred close to the learning rate reduction steps. This suggests that a smoother learning rate schedule, such as cosine annealing, may help produce more stable convergence and could potentially improve final test performance. A further methodological limitation is the absence of per-language-pair metrics. Although the overall results show strong cross-language performance, the lack of separate results for each language pair makes it difficult to measure exactly how well the system performs across specific combinations of programming languages. This should be addressed in future work through systematic per-pair evaluation.

6.6.4. Practical Implementation

The deployment of the system as a React and FastAPI web application shows that the proposed pipeline is not only a research prototype, but also a practically deployable tool. This is important because plagiarism detection systems must be used in real academic environments, not only evaluated through offline experiments. The IR pre-filtering stage improves the practical efficiency of the system by reducing the number of comparisons required during detection. This helps avoid the $O(n^2)$ complexity problem found in tools such as MOSS, where every submission may need to be compared against every other submission (Wan, Liu and Gao, 2018). As a result, the proposed system is more suitable for real-time or large-scale submission scenarios.

Another practical strength is the system's multi-layer output design. By presenting IR similarity, AST similarity, semantic similarity scores, and matched code regions, the system provides educators with clearer and more explainable evidence. This is more useful than giving only a

single numerical score, as it allows academic staff to make informed decisions rather than relying on an opaque automated verdict. This directly supports the transparency and accountability requirements discussed in Chapter 4.

6.7. Chapter Summary

This chapter presented a comprehensive evaluation of the proposed cross-language plagiarism detection system. The evaluation covered quantitative performance metrics, training behavior, confusion matrix analysis, qualitative case studies, ablation analysis, and comparison with similar systems in literature. The proposed system achieved an accuracy of 91.71%, precision of 93.42%, recall of 88.83%, F1 score of 91.06%, and AUC-ROC of 0.9683 on a held-out test set of 3,294 cross-language code pairs. These results exceed the 90% performance target defined in the research objectives and show that the system performs strongly when compared with existing approaches discussed in the literature. The training process converged over 14 epochs, with early stopping preserving the best-performing checkpoint at Epoch 7. The qualitative analysis further confirmed that the system is able to detect cross-language plagiarism cases that would not be visible to traditional token-based tools. At the same time, the system maintained a low false positive rate, which is important for responsible use in academic integrity contexts. However, the system still has limitations. The main limitation is the false negative rate of 11.17%, particularly in cases involving semantic obfuscation. This identifies an important direction for future improvement, especially through stronger semantic representation and better handling of structurally different but functionally equivalent code. Overall, the findings show that the proposed system is effective, practical, and suitable as a foundation for cross-language plagiarism detection. Chapter 7 presents the final conclusions of the research and provides recommendations for future work.

CHAPTER 7 – CONCLUSION AND FUTURE WORK

7.1. Chapter Overview

This chapter brings together the research carried out across the previous chapters into a clear set of conclusions. It begins by summarizing how each research objective and research question was addressed, while mapping the outcomes against the original aims of the study. The chapter then restates the theoretical and practical contributions of the research, acknowledges its limitations, and presents future research directions based on those limitations. Finally, the chapter closes with a reflection on the significance of this work and the future role of cross-language semantic plagiarism detection in academic integrity.

7.2. Similar Findings

This research aimed to address a clear gap in academic integrity tools: the inability of existing plagiarism detection systems to identify semantically equivalent code submissions across different programming languages. Three research questions and five SMART objectives were defined in Chapter 1. All five objectives were addressed, and all three research questions were answered affirmatively through the evaluation presented in Chapter 6.

7.2.1. Research Questions

RQ1: asked how IR similarity pre-filtering, Tree-sitter AST normalization, and UniXcoder embeddings could be combined to detect within-language plagiarism, including logic-preserving obfuscation.

The system's overall accuracy of 91.71% and false positive rate of only 5.67% (98 out of 1,728 non-plagiarized pairs) confirm that the pipeline handles both lexical and structural obfuscation effectively, outperforming the lexical baselines discussed in the literature.

RQ2: asked whether the pipeline could detect cross-language plagiarism without relying on language-specific syntactic features.

The AUC-ROC score of 0.9683 and the system architecture support this finding. Tree-sitter provides a uniform AST API across Java, Python, and C#, while UniXcoder is pre-trained on a multilingual code corpus. Together, these features confirm that language-agnostic semantic representations can successfully bridge the cross-language gap.

RQ3: asked how the proposed system performs when compared with established baselines.

The comparative analysis in Chapter 6 shows that the proposed system, with an F1 score of 91.06% and accuracy of 91.71%, outperforms MOSS (~79% F1), JPlag (~83% F1), ESA-GST (~71% F1), CodeBERT + AutoML (~87% F1), and the prior Siamese + AST approach (~85% F1).

7.3 Contributions

7.3.1 Contributions to the Research Domain

Novel pipeline architecture.

This research is the first to combine Tree-sitter multi-language AST normalization, UniXcoder semantic embeddings, and a Siamese contrastive neural network into a single end-to-end trainable pipeline for cross-language academic plagiarism detection.

No prior study had integrated these three components. Guo et al. (2022) proposed UniXcoder in isolation, while Fokam and Ajoodha (2021) applied a Siamese network only to monolingual code.

First structured cross-language academic dataset.

The research produced the first publicly documented and labelled dataset covering cross-language plagiarism pairs across Java, Python, and C#, derived from an academically validated corpus.

This directly addresses Gap 3 from Chapter 1 and provides a reusable evaluation testbed for future researchers.

Scalable candidate filtering design.

The IR pre-filtering stage addresses the $O(n^2)$ computational complexity problem found in existing tools (Wan, Liu and Gao, 2018).

The ablation study confirmed that removing it does not harm detection quality, but only affects computational efficiency. This establishes it as a practically important architectural pattern for future semantic detection systems at scale.

7.3.2 Contributions to the Problem Domain

Educator-accessible detection tool.

The full-stack web application provides a practical and deployable interface that requires no machine learning expertise to operate. This directly addresses the usability gap identified in Chapter 1 (Gap 4).

The multi-layer output, which includes IR score, AST similarity, semantic similarity, and matched code highlights, gives academic staff interpretable and evidence-backed triage rather than opaque numerical verdicts.

High-precision academic integrity support.

A precision score of 93.42% means that fewer than 6% of flagged submissions are false accusations. In an environment where wrongful plagiarism findings carry serious academic and personal consequences for students (British Computer Society, 2022), this precision level represents a meaningful practical improvement over the literature baselines.

Reduced educator workload.

By automating the triage of submission pairs and surfacing only high-confidence candidates for human review, the system directly reduces the manual assessment burden identified by Aniceto, Holanda and Da Silva (2023) in large programming cohorts.

7.4 Limitations

False negative rate of 11.17%.

The most significant limitation is that 175 genuinely plagiarised pairs in the held-out test set were not detected. Analysis of failure cases points to semantic obfuscation as the primary cause. This occurs when students replace an algorithm with a functionally equivalent but structurally different solution. UniXcoder embeddings partially encode implementation strategy, not only computational outcome. This means that different-strategy implementations of the same function can be mapped to distant vectors, even when the underlying logic is equivalent.

No per-language-pair evaluation.

Although the overall metrics are strong, Chapter 6 acknowledges the absence of separate precision and recall breakdowns for the Java–Python, Java–C#, and Python–C# pair types. This prevents the identification of any weaker language combination, which is important for ensuring equitable detection across student cohorts, as flagged in Chapter 4.

Dataset scope.

The dataset is derived from a single introductory-level Java corpus and covers only three programming languages. Generalization to advanced algorithms, different assignment types, or additional languages has not been validated.

No live institutional deployment.

The system was tested on a constructed dataset, not on live student submissions from a real institution. Ecological validity, including real-world obfuscation diversity, genuine plagiarism intent, and instructor workflow integration, remains to be established.

7.5 Future Work

The limitations discussed above directly lead to the following future research directions.

Per-Language-Pair Evaluation and Threshold Calibration

An immediate priority for future work is to report precision, recall, and F1 score separately for each language combination. This would allow language-specific threshold calibration and help address both the fairness concern discussed in Chapter 4 and the analytical gap identified in Chapter 6.

Improved Semantic Obfuscation Handling

The false negative rate of 11.17% is mainly caused by semantic obfuscation. Future work should explore graph-based code representations, such as Program Dependency Graphs (PDGs) or Control Flow Graphs (CFGs), as inputs to the embedding stage. These graph structures capture computational intent more directly than linearized token sequences and are less sensitive to variations in implementation strategy.

Expansion to Additional Programming Languages

Extending the pipeline to JavaScript, Rust, Go, and Kotlin would reflect modern computer science curricula more broadly. It would also test whether the language-agnostic design generalizes as intended beyond the three programming languages evaluated in this research.

AI-Generated Code Detection Integration

As noted in Chapter 1, AI code generators produce structurally diverse outputs that share the obfuscation profile of cross-language plagiarism. Extending the Siamese network to distinguish AI-generated submissions from human-authored ones, potentially using contrastive training on AI-generation provenance labels, would make the tool more relevant to the rapidly evolving academic integrity landscape described by Sağlam and Schmid (2025).

LMS Integration

Integration with Learning Management Systems such as Moodle or Canvas through a plugin architecture would embed detection into existing institutional workflows. This would reduce the friction of using a standalone submission portal and increase adoption among non-technical educators.

7.6 Contribution to the Body of Knowledge

This research advances the body of knowledge in two dimensions, which directly correspond to the four gaps identified in Chapter 1.

7.6.1. Research Domain

The proposed pipeline, which combines IR pre-filtering, Tree-sitter AST normalization, UniXcoder embedding, and Siamese contrastive classification, represents a novel methodological contribution that no prior study had formulated or evaluated.

The results, with an F1 score of 91.06% and an AUC-ROC of 0.9683 on a 3,294-pair cross-language test set, provide the first empirically validated and replicable baseline specifically for cross-language academic plagiarism detection. Future methods can build on this baseline and aim to improve it further.

The constructed dataset also fills a structural gap in the field, which Karnalim et al. (2019) and Ebrahim and Joy (2023) each identified as a major obstacle to progress.

7.6.2. Problem Domain

The deployed web application demonstrates that moving from a research prototype to a usable educator-facing tool is achievable within the constraints of a single-developer academic project.

The high-precision and explainable output design applies the ethical and professional principles discussed in Chapter 4. It reflects the BCS Code of Conduct requirements for transparency, accountability, and human-in-the-loop decision-making within a concrete and accessible product.

Therefore, this work contributes not only a detection model, but also to reference architecture for the responsible deployment of machine-learning-based academic integrity systems.

7.7 Concluding Remarks

This project began with a clearly defined and practically important goal: to build a plagiarism detection system capable of identifying semantically equivalent code across Java, Python, and C#. This is a capability that current production tools still lack. Every research objective was met, the 90% F1 performance target was surpassed, and the system was delivered as a working web application accessible to educators within the June 2026 deadline.

The 11.17% false negative rate is the clearest indication of where the work should go next, and the future directions outlined above translate that limitation into a concrete research roadmap. Academic integrity tooling is now at an important turning point. The combination of multi-language curricula, AI-assisted code generation, and increasingly sophisticated obfuscation strategies means that lexical and monolingual approaches are no longer able to keep pace.

This research demonstrates that language-agnostic semantic representations, supported by pre-trained multilingual code models and trained using metric learning objectives, offer a principled and empirically validated path forward. The system and dataset produced in this research are not intended to be endpoints, but a reproducible foundation for future work. This foundation can support expansion to more programming languages, richer obfuscation taxonomies, and real-world institutional deployments, all in support of fairer and more accurate academic assessment.

REFERENCES

Aiken, A. (1994). MOSS: A system for detecting software plagiarism. [online] Available at: https://theory.stanford.edu/~aiken/moss/.

Aniceto, C.R., Holanda, M. and Silva, D. (2023). Automating source code plagiarism detection in a moodle-based programming course. pp.1–7. doi:https://doi.org/10.1109/fie58773.2023.10342895.

British Computer Society (2022). *BCS code of conduct*. [online] Available at: https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/.

Ebrahim, F. and Joy, M. (2023). Source code plagiarism detection with pre-trained model embeddings and automated machine learning. doi:https://doi.org/10.26615/978-954-452-092-2_034.

Fokam, Fabrice M and Ajoodha, R. (2021). Influence of contrastive learning on source code plagiarism detection through recursive neural networks. doi:https://doi.org/10.1109/imitec52926.2021.9714688.

Foltýnek, T., Všianský, R., Meuschke, N., Dlabolová, D. and Gipp, B. (2020). Cross-language source code plagiarism detection using explicit semantic analysis and scored greedy string tiling. doi:https://doi.org/10.1145/3383583.3398594.

Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M. and Yin, J. (2022). UniXcoder: Unified cross-modal pre-training for code representation. [online] pp.7212–7225. Available at: https://aclanthology.org/2022.acl-long.499.pdf.

Karnalim, O., Budi, S., Toba, H. and Joy, M. (2019). Source code plagiarism detection in academia with information retrieval: Dataset and the observation. *Informatics in Education*, 18, pp.321–344. doi:https://doi.org/10.15388/infedu.2019.15.

King, M.R. (2023). A conversation on artificial intelligence, chatbots, and plagiarism in higher education. *Cellular and Molecular Bioengineering*, 16. doi:https://doi.org/10.1007/s12195-022-00754-8.

Lu, X. and K, W.H. (2025). The application of code plagiarism detection function in a c programming university course. pp.1–4. doi:https://doi.org/10.1109/is3c65361.2025.11130958.

Maertens, R., Petegem, V., Strijbol, N., Baeyens, T., Jacobs, A.C., Dawyndt, P. and Mesuere, B. (2022). Dolos: Language-agnostic plagiarism detection in source code. *Journal of Computer Assisted Learning*, 38, pp.1046–1061. doi:https://doi.org/10.1111/jcal.12662.

Office, I.C. (2021). *Guide to the UK general data protection regulation (UK GDPR)*. [online] Available at: https://ico.org.uk/for-organisations/guide-to-data-protection/guide-to-the-general-data-protection-regulation-gdpr/.

Prechelt, L., Malpohl, G. and Philippsen, M. (2002). Finding plagiarisms among a set of programs with JPlag. In: *Journal of Universal Computer Science*. [online] pp.1016–1038. Available at: https://doi.org/10.3217/jucs-008-11-1016.

Sağlam, T. and Schmid, L. (2025). *Evaluating software plagiarism detection in the age of AI: Automated obfuscation and lessons for academic integrity*. [online] arXiv preprint. Available at: https://arxiv.org/abs/2505.20158.

Santos, R. and Ferreira, J.D. (2020). Plagiarism detection based on blinded logical test automation results and detection of textual similarity between source codes. pp.1–9. doi:https://doi.org/10.1109/fie44824.2020.9274098.

Saunders, M., Lewis, P. and Thornhill, A. (2019). *Research methods for business students*. 8th ed. Pearson.

Tree-sitter (2023). *Tree-sitter: An incremental parsing system for programming tools*. [online] Available at: https://tree-sitter.github.io/tree-sitter/.

Wan, H., Liu, K. and Gao, X. (2018). Token-based approach for real-time plagiarism detection in digital designs. doi:https://doi.org/10.1109/FIE.2018.8658531.

Zakeri-Nasrabadi, M., Parsa, S., Ramezani, M., Roy, C. and Ekhtiarzadeh, M. (2023). *A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges*. [online] arXiv preprint. Available at: https://arxiv.org/abs/2306.16171.

APPENDICES

Gantt Chart

ALL	I...	TASK NAME	DURATION	PLANNED STA...	PLANNED FINIS...
1		**Project Planning & Initiation**	**79 days**	**2025-07-02**	**2025-10-21**
2		Idea Research	28 days	2025-07-02	2025-08-08
3		Tentative Topic Submission	1 day	2025-08-11	2025-08-11
4		Finalization of Supervisor	1 day	2025-08-12	2025-08-12
5		Problem Identification	17 days	2025-07-21	2025-08-12
6		Existing Work related to Idea	37 days	2025-07-21	2025-09-09
7		Initial Research Proposal Submissio...	1 day	2025-09-15	2025-09-15
8		Final Project Proposal	0 days	2025-10-21	2025-10-21
9			1 day	2025-10-27	2025-10-27
10		Literature Review	0 days	2025-11-24	2025-11-24
11		Collecting Research Papers	12 days	2025-07-02	2025-07-17
12		Evaluating the related works	13 days	2025-07-02	2025-07-18
13		Identifying and evaluating research gaps	10 days	2025-07-08	2025-07-21
14		Literature Review Submission	1 day	2025-10-21	2025-10-21
15		**SRS**	**29 days**	**2025-10-28**	**2025-12-05**
16		Identifying the Requirements	10 days	2025-10-28	2025-11-10
17		Preparing and conducting surveys	12 days	2025-11-05	2025-11-20
18		Analysing the gathered Data	2 days	2025-11-21	2025-11-24
19		Preparing SRS document	6 days	2025-11-24	2025-12-01
20		Supervisor reviews and gives feedb...	2 days	2025-12-02	2025-12-03
21		Final SRS submission	1 day	2025-12-05	2025-12-05
22		**Proof Of Concept**	**14 days**	**2025-12-12**	**2025-12-31**
23		Identifying & Analysing current tech...	4 days	2025-12-12	2025-12-17
24		Identifying the limitations in existing...	6 days	2025-12-18	2025-12-25
25		Document Writing	3 days	2025-12-26	2025-12-30
26		POC Submission	1 day	2025-12-31	2025-12-31

27	Mid Point Submission	1 day	2026-01-27	2026-01-27
28	Gathering deliverables for Mid-Point	3 days	2026-01-05	2026-01-07
29	Documenting the functional & Non-fun...	4 days	2026-01-07	2026-01-12
30	Implementing main function of the syst...	31 days	2026-01-19	2026-03-02
31	Supervisor Reviews the Document	2 days	2026-01-15	2026-01-16
32	Refining the Document	8 days	2026-01-16	2026-01-27
33	Testing main features	10 days	2026-03-03	2026-03-16
34	MVP Submission	1 day	2026-03-24	2026-03-24
35	**Thesis Submission**	**54 days**	**2026-04-02**	**2026-06-16**
36	Gathering and Implementing feedba...	5 days	2026-04-02	2026-04-08
37	Conducting & Updating Literature R...	33 days	2026-04-09	2026-05-25
38	Completing Documentation	8 days	2026-05-25	2026-06-03
39	Supervisor Reviews the Document	3 days	2026-06-04	2026-06-08
40	Refining the document	5 days	2026-06-10	2026-06-16
41	Thesis Submission	1 day	2026-06-16	2026-06-16